

A Real-time Research Project Report

**EMPOWERING REAL TIME ECONOMIC DECISION THROUGH
EDGE COMPUTING FOR MINIMAL LATENCY**

Submitted in partial fulfilment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

**COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE &
MACHINE LEARNING)**

By

BATCH: 03

M. BHARATH (23UJ1A6683)

B. ASHOK KUMAR (23UJ1A6662)

K. SANTOSH (23UJ1A6676)

V. CHARAN KUMAR (23UJ1A66A0)

S.GNANESWAR (23UJ1A6696)

Under the esteemed guidance of

Dr. B. RAMESH BABU PhD.

Assistant Professor

Department of Computer Science and Engineering (Artificial Intelligence & Machine
Learning

At



MALLA REDDY ENGINEERING COLLEGE AND MANAGEMENT SCIENCES

Kistapur (V), Medchal (M), Medchal Malkajgiri (Dist)-501401.

(An UGC autonomous Institution, NBA Accredited in CSE, ECE, IT, EEE)

2023-2027

Affiliated to



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD

KUKATPALLY, HYDERABAD-85



MALLA REDDY ENGINEERING COLLEGE AND MANAGEMENT SCIENCES

Kistapur (V), Medchal (M), Medchal Malkajgiri (Dist)-501401.

(An UGC autonomous Institution, NBA Accredited in CSE, ECE, IT, EEE)

CERTIFICATE

This is to certify that the Real-time Research Project report entitled “Empowering Real-Time Economic Decision Through Edge Computing for Minimal Latency” is the bonafide work carried out and submitted by

M. BHARATH (23UJ1A6683)

B. ASHOK KUMAR (23UJ1A6662)

K. SANTOSH (23UJ1A6676)

V. CHARAN KUMAR (23UJ1A66A0)

S.GNANESWAR (23UJ1A6696)

To the department of

Computer Science and Engineering (Artificial Intelligence & Machine Learning), Malla Reddy Engineering College and Management Sciences, in partial fulfilment for the award of BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING) during the academic year 2024-2025.

Internal Guide

Dr. B. Ramesh Babu PhD.

Assistant Professor

Department of CSE

(Artificial Intelligence and Machine Learning)

Head of The Department

Dr. G. RADHA DEVI PhD.

Assistant Professor

Department of CSE

(Artificial Intelligence and Machine Learning)

EXTERNAL

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible, whose constant guidance and encouragement crowned our efforts with success. It is a pleasant aspect that we have now the opportunity to express my guidance for all of them.

We are thankful to **Mr.V.MALLA REDDY** chairman of Malla Reddy Engineering College and Management Sciences for accepting ours and providing us with an opportunity to do a project in their esteemed organization.

We are thankful to Principal **Dr.M.SREEDHAR REDDY** M.Tech., Ph.D. Malla Reddy Engineering College and Management Sciences for helped ours to undergo project work as a part of university curriculum.

We are special thanks to **Dr. RADHA DEVI** M.Tech,Ph.D.. Assistant Professor& Head in Computer Science and Engineering (AI & ML) Department and **Dr. B. Ramesh Babu** PhD Assistant Professor in Computer Science and Engineering (AI & ML) Department for guiding us in the right way to complete our project in the right time.

We would like to thank our internal project mates anud department faculties for their full-fledged guidance and giving courage to carry out the project.

We are very much thankful to one and all that helped us for the successful completion of our project.

M. BHARATH	(23UJ1A6683)
B. ASHOK KUMAR	(23UJ1A6662)
K. SANTOSH	(23UJ1A6676)
V. CHARAN KUMAR	(23UJ1A66A0)
S.GNANESWAR	(23UJ1A6696)

TABLE OF CONTENT

	Page Numbers
List of Figures	i
List of Tables	i
ABSTRACT	1
CHAPTER 1: INTRODUCTION	2-4
1.1 Overview	2
1.2 Research Motivation	3
1.3 Problem Definition	3
1.4 Significance	3
1.5 Applications	4
CHAPTER 2: LITERATURE SURVEY	5-29
2.1 Introduction	5
2.2 Related Work	5-28
2.3 Research Gaps	28
2.4 Summary	28-29
CHAPTER 3 : EXISTING SYSTEM	30-31
3.1 Centralized Financial Decision Making	30
3.2 Traditional Retail Management	30-31
3.3 Municipal Economic Planning in Smart Cities	31
3.4 Drawbacks of Manual Systems	31

CHAPTER 4 : PROPOSED METHODOLOGY	32-43
4.1 System Architecture	32-35
4.2 Data Preprocessing	35-36
4.3 Exploration data analysis	36-38
4.4 Dataset Splitting	38-39
4.5 Model Building and Training	39-43
CHAPTER 5 : UML DIAGRAMS	44-49
5.1 Class Diagram	45
5.2 Data Flow Diagram	46
5.3 Sequence Diagram	47
5.4 Activity Diagram	47
5.5 Deployment Diagram	48
5.6 Use Case Diagram	48
5.7 Component Diagram	49
CHAPTER 6 : SOFTWARE REQUIREMENTS	50-53
6.1 Software Requirements	50-51
6.2 Hardware Requirements	52-53
CHAPTER 7 : FUNCTIONAL REQUIREMENTS	54-56
CHAPTER 8 : SOURCE CODE	57-66

CHAPTER 9: RESULTS & DICUSSIONS	67-75
9.1 Dataset Description	69-70
9.2 Result Analysis	70-74
9.3 Comparative Analysis	74
CHAPTER 10: CONCLUSION AND FUTURE SCOPE	75
10.1 Conclusion	75
10.2 Future Scope	75
REFERENCES	76-80

LIST OF FIGURES

Fig. 1.1: Applications of Edge Computing in IoT-Driven Smart Environments.	2
Fig. 4.1: Architecture for Real Time Economic Decision via Edge Computing.	33
Fig. 4.5(a): Block diagram of working flow of Extra Trees Classifier.	40
Fig. 4.5(c): Workflow of Proposed Cat Boost Classifier.	42
Fig. 5.1: Class Diagram	45
Fig. 5.2: Dataflow Diagram	. 46
Fig. 5.3: Sequence Diagram.	47
Fig. 5.4: Activity Diagram.	47
Fig. 5.5: Deployment Diagram.	48
Fig. 5.6: Use Case Diagram.	48
Fig. 5.7: Component Diagram	49
Fig. 9.2(a): Sample Dataset	71
Fig. 9.2(b): Correlation Matrix of System and Market Parameters	71
Fig. 9.2(c): Distribution of Decision Outcome Categories	72
Fig. 9.2(d): Transaction Volume Distribution by Decision Outcome	72
Fig. 9.2(e): Financial Metric vs System Throughput by Decision Outcome.	73
Fig. 9.2(f): Distribution of Resource Utilization.	73
Fig. 9.2(g) : Predictions on test data.	74

LIST OF TABLE

Table. 9.1: Comparison of existing Extra Trees and Cat Boost Classifier.	74
--	----

EMPOWERING REAL TIME ECONOMIC DECISION THROUGH EDGE COMPUTING FOR MINIMAL LATENCY

Abstract

Real-time economic decision-making is becoming increasingly critical, especially as over 60% of global enterprises seek faster data insights, and edge computing is projected to reach a market value of \$155 billion by 2025. Despite this growth, many economic systems still rely on manual or centralized computing methods, which suffer from high latency and inefficient decision loops, specially under dynamic conditions. The project presents a novel edge-computing powered pipeline for accelerating economic decision outcomes using intelligent machine learning. The contribution begins with robust data preprocessing, where missing values are handled, target variables are encoded, and features are standardized for consistent input. A comprehensive Exploratory Data Analysis (EDA) is performed to uncover relationships between key economic indicators such as transaction volume, system throughput, and resource utilization. The dataset is then split into training and testing sets to facilitate reliable model evaluation. Among classification techniques, an Extra Trees Classifier is adopted as an existing benchmark due to its ensemble accuracy, while the Cat Boost Classifier is proposed as a novel solution for this task, offering gradient boosting with categorical handling and superior performance on tabular data. The Cat Boost model demonstrates enhanced accuracy, precision, recall, and F1-score in predicting whether an economic decision outcome is effective or not. This integration of automated learning with edge-oriented design lays the foundation for real-time, low-latency, and high-accuracy decision support systems in economic management applications.

CHAPTER 1

INTRODUCTION

1.1 Overview

In the rapidly evolving landscape of economic systems, the need for real-time decision-making has become more pressing than ever. With the explosion of data generated by financial institutions, trading platforms, and decentralized economic networks, traditional centralized processing methods often fall short in delivering timely insights. Delays caused by network latency, bandwidth limitations, and computational overheads can result in missed opportunities and ineffective decision-making. Edge computing offers a transformative solution by enabling data processing closer to the source, thus minimizing latency and improving the responsiveness of economic applications.

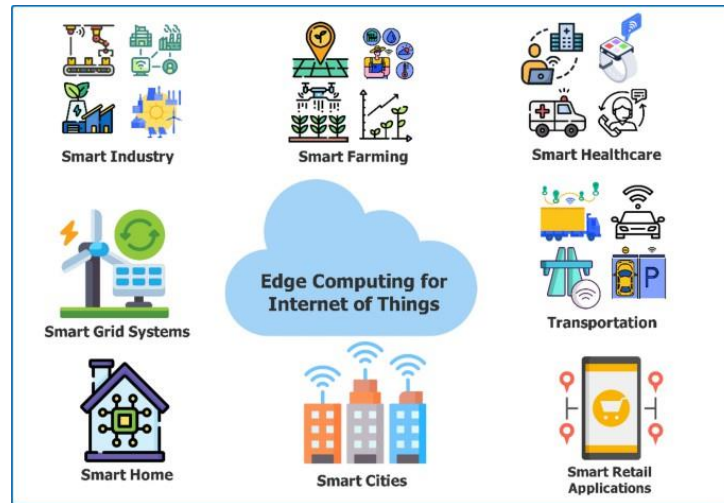


Fig. 1.1: Applications of Edge Computing in IoT-Driven Smart Environments.

The project, titled, Empowering Real-Time Economic Decision Through Edge Computing for Minimal Latency leverages modern machine learning techniques to classify economic decision outcomes efficiently. The process begins with structured data preprocessing, including handling missing values, scaling features, and encoding the target variable. Exploratory Data Analysis (EDA) is conducted to identify hidden trends and correlations among variables such as transaction volume, resource utilization, and financial metrics. The dataset is then split for training and evaluation. Among classifiers, Extra Trees serves as a reliable benchmark, while CatBoost is proposed for its efficiency and superior performance on structured data.

1.2 Research Motivation

In today's data-driven world, the speed and accuracy of decision-making are critical, especially in economic systems that operate in dynamic, high-stakes environments. Traditional centralized computing models often struggle to meet the low-latency demands of real-time applications due to network congestion, server bottlenecks, and long data transmission cycles. This has created a pressing need for decentralized solutions that bring computation closer to the data source. Edge computing addresses this challenge by offering localized data processing, significantly reducing latency and enabling faster responses. The integration of edge computing with machine learning opens new possibilities for intelligent economic systems capable of making rapid, data-informed decisions. However, building a robust decision-making model requires addressing challenges in data quality, feature selection, and model generalizability. This research is motivated by the potential to bridge this gap through a structured machine learning pipeline. By incorporating effective data preprocessing, insightful exploratory data analysis, and advanced classifiers, we aim to enhance the predictive accuracy of economic decisions. Our comparative study between existing models like Extra Trees and advanced models like CatBoost focuses on improving prediction outcomes while maintaining minimal processing time.

Problem Definition

In fast-paced economic environments, timely and accurate decision-making is often hindered by high latency and reliance on centralized computing systems. Traditional models struggle to process and analyze large volumes of real-time data efficiently, leading to delayed or suboptimal decisions. This research addresses the problem of predicting economic decision outcomes with minimal latency by leveraging edge computing and machine learning. The aim is to develop a lightweight, accurate, and scalable model that can process data near the source, reduce delays, and deliver real-time insights. The focus is on comparing traditional classifiers with advanced models to optimize prediction performance in edge environments.

1.3 Significance

- Enables real-time economic decision-making by minimizing latency through edge computing.
- Enhances prediction accuracy using advanced machine learning models like CatBoost.
- Improves system scalability and responsiveness in dynamic economic environments.
- Reduces dependency on centralized servers, lowering bandwidth and infrastructure costs.
- Supports intelligent automation across sectors such as finance, retail, and smart cities.

1.4 Applications

- **Smart Financial Services**

Edge computing enables real-time fraud detection, portfolio management, and transaction analysis by processing financial data close to the source. Integrating intelligent classifiers like CatBoost allows institutions to predict the effectiveness of financial decisions with high accuracy. This reduces response times and supports instant decision-making in dynamic markets, enhancing customer experience and financial security.

- **Smart Retail Analytics**

Retail environments can benefit from edge-enabled systems that analyze customer behavior, inventory flow, and sales patterns in real time. The proposed decision outcome model can assist in dynamic pricing, personalized offers, and stock optimization. By processing data at the edge, retailers can react to customer needs instantly, leading to improved service quality and profitability.

- **Smart City Management**

In smart cities, economic and infrastructural decisions—such as resource allocation, public service deployment, or energy distribution—require quick and data-driven insights. Using edge computing, cities can process data locally and apply machine learning models to assess the effectiveness of these actions, optimizing urban operations with minimal delay.

- **Intelligent Transportation Systems**

Economic decisions in transportation, like dynamic tolling or fleet management, benefit from real-time analytics. Edge computing reduces latency in data processing from sensors and traffic systems. By evaluating the predicted effectiveness of these decisions, transportation systems can ensure smoother traffic flow and cost-efficient resource usage.

- **Industrial Automation**

In smart industries, edge-based predictive models help evaluate decisions related to production optimization, energy consumption, and logistics. Deploying such models locally enables fast, automated adjustments in manufacturing processes. This not only improves operational efficiency but also minimizes downtime and maximizes throughput in competitive industrial settings.

CHAPTER 2

LITERATURE SURVEY

2.1 INTRODUCTION

The rapid growth of data-driven systems across various sectors—such as finance, energy, retail, agriculture, and manufacturing—has necessitated the development of technologies capable of supporting real-time, intelligent decision-making. Traditional cloud-based computing models, while powerful, often fall short when ultra-low latency, local responsiveness, and continuous availability are critical. This has led to the rise of edge computing, a paradigm that brings computational resources closer to the data source, enabling faster, context-aware decision-making with reduced reliance on centralized infrastructure.

Edge computing plays a transformative role in economic systems, where decisions must often be made instantly based on dynamic market conditions, user behaviours, or environmental inputs. Examples include real-time pricing in e-commerce, instant bidding in financial markets, smart grid energy management, and adaptive logistics in transportation. In such applications, delays even of milliseconds can lead to economic inefficiencies, missed opportunities, or increased operational costs.

2.2 RELATED WORK

PremSankar et al. [1] Edge Computing for the Internet of Things: A Case Study

This paper studies the application of edge computing in IoT environments. It focuses on reducing latency and enhancing real-time decision-making. The authors present a smart city use case to demonstrate these benefits. Results show significant performance improvements by processing data closer to the source. The study highlights edge computing's role in improving responsiveness.

Challenges such as resource constraints and scalability are acknowledged. Overall, the paper validates edge computing as crucial for real-time IoT applications.

Bjorkdahl et al. [2] Strategies for Digitalization in Manufacturing Firms

This study explores digitalization strategies within manufacturing firms. It emphasizes the growing importance of edge computing in Industry 4.0 transformations. The author identifies investment in IoT and edge technologies as key strategies. The paper discusses how these technologies support

automation and data-driven decision-making. It also addresses organizational and technological challenges in digital adoption. Aligning technology initiatives with business goals is stressed for success. The study provides practical insights for effective digital transformation.

Stojmenovic [3] Fog Computing: A Cloud to the Ground Support for Smart Things and Machine-to-Machine Networks. This paper introduces fog computing, which extends cloud computing closer to IoT and M2M devices. It aims to reduce latency and improve real-time communication. Fog computing supports distributed processing at the network edge, bridging the gap between cloud and devices. The author explores its applications in smart devices and industrial networks. Benefits include enhanced responsiveness and scalability. Fog computing helps overcome cloud limitations for latency-sensitive tasks. The paper emphasizes fog as a critical layer in IoT architecture. It highlights improved data processing closer to data sources. The concept supports more efficient and timely decision-making.

Oh et al. [4] A Product Quality Prediction Model Using Real-Time Process Monitoring in Manufacturing Supply Chain. This study develops a predictive model based on real-time process monitoring data. It aims to predict product quality and reduce defects in manufacturing. The model leverages data collected continuously from production processes. Authors show the model's effectiveness in enhancing manufacturing reliability. Real-time monitoring is crucial for early defect detection. The paper demonstrates integration of edge computing in industrial environments. This approach supports Industry 4.0 quality management initiatives. It improves process transparency and control. Overall, it enhances product consistency and reduces waste.

Zhu et al. [5] Research on the Application of Real-Time Monitoring System for Manufacturing Quality of Industrial Production Based on Industrial 4.0. This paper explores real-time monitoring systems to enhance manufacturing quality. It highlights the role of edge computing and IoT technologies in Industry 4.0. The author shows how real-time data improves defect detection and process control. The system reduces production errors and boosts efficiency. Industrial 4.0 technologies enable seamless integration and automation. Benefits include faster response to quality issues and better resource utilization. The paper underscores real-time monitoring's impact on manufacturing competitiveness. It supports smart factory implementations. It promotes continuous improvement in industrial production.

Jiao et al. [6] Cloud-Based Computation Offloading for Mobile Devices: State of the Art, Challenges and Opportunities. This review paper covers cloud-based computation offloading techniques for mobile devices. It discusses benefits such as reducing latency and conserving device energy. The authors analyze challenges like network reliability and security risks. They survey different architectures and offloading algorithms. The paper identifies opportunities to enhance mobile computing performance. Cloud offloading allows resource-intensive tasks to be handled remotely. This reduces mobile device constraints and improves user experience. The study highlights future research directions. It bridges mobile and cloud computing effectively.

Barboni et al. [7] Machine Learning and Edge Computing for Industry 4.0 Applications: Concepts and Extensive Review. This comprehensive review examines combining machine learning with edge computing in Industry 4.0. It emphasizes real-time data processing near production sites for faster decision-making. The paper highlights benefits such as improved automation and predictive maintenance. Authors cover various industrial applications and case studies. Challenges include computational limits on edge devices and data privacy. The review discusses solutions and future trends. It promotes integrating AI and edge for smart manufacturing. The paper supports digital transformation strategies. It showcases enhanced operational efficiency through technology synergy.

Willner and Gowtham et al. [8] Toward a Reference Architecture Model for Industrial Edge Computing. This paper proposes a standardized reference architecture model for industrial edge computing. The model aims to improve interoperability and deployment consistency. It addresses challenges related to resource management and system scalability. Benefits include easier integration of diverse industrial components. The authors demonstrate how the model supports efficient edge infrastructure. It facilitates flexible and scalable industrial IoT solutions. The architecture promotes real-time processing and reduced latency. It acts as a blueprint for industry-wide edge computing adoption. The paper supports streamlined industrial digitalization.

Veith et al. [9] Latency-Aware Placement of Data Stream Analytics on Edge Computing.

This study investigates strategies for placing data stream analytics on edge devices with latency-awareness. The goal is to minimize processing delays and enhance real-time responsiveness. Authors propose algorithms for optimal placement of analytics tasks. Results show significant latency reduction and improved performance. The approach helps meet stringent time requirements of edge applications. It improves utilization of edge computing resources. The paper discusses implications

for time-critical IoT and industrial systems. Latency-aware placement supports efficient and reliable analytics. This research contributes to optimizing edge-based data processing systems.

Aral et al. [10] propose an edge scheduling mechanism to effectively address application latency requirements in edge computing environments. The scheduling system dynamically allocates computational resources based on task priorities and time-sensitivity. Their model demonstrates how strategic task scheduling can reduce delays in data processing. It enhances performance and responsiveness in latency-sensitive applications, particularly in real-time IoT and industrial use cases. The approach is adaptive to varying network conditions and workload patterns. Results from simulations confirm that their method significantly improves system throughput. The study provides a foundation for real-time edge orchestration. It bridges the gap between application needs and infrastructure constraints. This work supports more responsive and efficient edge services.

Zhang et al. [11] introduce a real-time task reconfiguration mechanism tailored for industrial edge computing environments. Their proposed model supports dynamic reassignment of computational tasks to optimize system performance. The mechanism ensures low latency and high availability even under fluctuating workloads. It proves particularly useful for time-critical industrial operations. Their framework enhances processing speed while maintaining system stability. Evaluation results highlight improvements in efficiency and responsiveness. The method allows for intelligent load balancing and resource optimization. It supports fault-tolerant edge computing for manufacturing. This contribution is crucial for enabling flexible and adaptive industrial systems.

Bacchiani et al. [12] In the SEAWALL case study, Bacchiani et al. explore low-latency anomaly detection using a combined edge-cloud architecture. Their solution monitors industrial processes in real time to detect abnormal patterns. The hybrid model leverages the strengths of both edge and cloud computing to ensure timely alerts. Local edge processing minimizes delay in detection, enhancing decision-making speed. The cloud provides computational support for more complex analytics. Their approach effectively reduces system vulnerabilities and production faults. It improves reliability and operational awareness in Industry 4.0 settings. The study underscores the importance of placing critical detection tasks at the edge. It serves as a benchmark for real-time fault management systems.

Abouaomar et al. [13] investigate resource provisioning strategies for latency-sensitive applications in edge computing. Their study presents an adaptive provisioning model that responds to the changing demands of applications in real time. It aims to reduce latency by efficiently distributing computational and storage resources. Their solution enhances service continuity and maintains performance standards under variable network loads. The system dynamically monitors and allocates resources where needed most. Simulation results show improved throughput and reduced service delay. This approach is especially suitable for smart environments and time-critical systems. It ensures optimal usage of edge infrastructure. The model is a step forward in agile and responsive edge service management.

Jo et al. [14] benchmark GPU-accelerated edge devices to evaluate their performance and energy efficiency. The study compares different hardware setups in handling intensive workloads, especially in machine learning and data processing tasks. GPU-enabled edge devices show considerable advantages in execution speed and parallelism. The research highlights how GPU acceleration can improve real-time responsiveness in edge applications. It also addresses power consumption trade-offs in various device configurations. Their findings offer guidance on selecting edge hardware based on workload types. The study supports optimized deployment of AI workloads at the edge. It reinforces the role of hardware in edge efficiency. This benchmark is valuable for developers and system architects.

Hussain and Salehi [15] This study by Hussain and Salehi focuses on resource allocation for Industry 4.0 micro-services in a serverless fog federation environment. The proposed model efficiently distributes micro-services across multiple fog nodes. It ensures optimal resource use and reduces deployment latency. Their approach supports scalable and flexible service orchestration in distributed architectures. The study improves the availability and performance of industrial services. The allocation method responds dynamically to workload changes. It enhances operational efficiency without the overhead of central management. Their work contributes to resilient and low-latency service deployment. This model is essential for Industry 4.0 environments requiring fast and flexible computing.

Shi et al. [16] provide a comprehensive review of the vision, architecture, and challenges of edge computing. The paper discusses how edge computing differs from traditional cloud models by offering low-latency and context-aware services. The authors highlight key benefits such as improved scalability, faster response times, and better bandwidth use. However, they also identify challenges like

security, mobility, and resource management. The review explores use cases across healthcare, smart cities, and autonomous systems. It emphasizes the need for robust infrastructure and intelligent orchestration. Their work sets the stage for future edge computing research. It is widely cited in foundational edge computing literature. The study outlines a roadmap for system architects and researchers.

Satyanarayanan et al. [17] examine the emergence and significance of edge computing through a detailed literature review. Their work articulates how edge computing supports real-time applications by processing data closer to the source. The authors describe its role in enhancing user experiences and enabling intelligent automation. The study discusses both technological advancements and deployment challenges. It focuses on how edge computing benefits mobile applications and IoT systems. The paper also considers issues such as security, heterogeneity, and performance management. Their analysis is supported by numerous real-world applications. The study helped shape the early direction of edge computing research. It remains influential in understanding edge paradigm shifts.

Varghese et al. [18] conducted a comprehensive review exploring the challenges and opportunities in edge computing. The study emphasizes edge computing's potential to bring computation closer to data sources, enabling faster processing. The authors identify several key benefits, such as reduced latency, bandwidth optimization, and enhanced scalability. However, the study also addresses challenges like resource constraints, heterogeneous environments, and security concerns. Their review stresses the need for efficient orchestration and interoperability across edge systems. It outlines the trade-offs between centralized and decentralized models. The authors also discuss use cases in smart cities and industrial IoT. Their work provides foundational insights for future edge research. This study is a valuable reference for system designers and researchers.

Hu, Youg et al. [19] explore mobile edge computing (MEC) as a fundamental enabler for next-generation 5G networks. The authors review the existing literature to identify MEC's impact on reducing network congestion and improving latency. The study highlights how MEC supports real-time services, including autonomous vehicles and augmented reality. It discusses architectural innovations and integration strategies with 5G infrastructure. Key benefits include localized content delivery and enhanced user experience. Challenges such as mobility management and dynamic resource allocation are also analyzed. The authors emphasize the importance of context-aware computing. Their findings underscore MEC's role in realizing 5G's low-latency promise.

Dinh et al. [20] present a detailed survey of mobile cloud computing (MCC), focusing on architecture, applications, and challenges. Their review investigates how MCC allows resource- constrained mobile devices to offload tasks to the cloud. The study outlines benefits such as improved computational power, extended battery life, and enhanced application performance. The authors also explore challenges like data privacy, latency, and dependency on network availability. Applications in healthcare, mobile learning, and location-based services are discussed. The paper compares different MCC models, such as augmentation and virtualization. It highlights the importance of seamless integration between mobile and cloud components. Their analysis contributes to optimizing mobile-cloud collaboration. The study offers direction for efficient mobile-cloud frameworks.

Yi, Shanhe et al. [21] explore how fog computing works and where it can be used. Fog computing is like a mini-computer placed near the user or device, which helps process data faster without sending it far away to a big server. The authors looked at many earlier studies to find out what's good and what's difficult about using this technology. They used a method that focuses on understanding ideas rather than numbers. Their research showed that fog computing is helpful in IoT (Internet of Things), where many smart devices are connected. It reduces delays by processing information closer to the devices. This means smart systems, like traffic lights or health monitors, can react quickly. The study also highlights how fog computing makes real-time decision-making better and more efficient.

Mahmoud, Magdy et al. [22] study how to make edge computing more secure. Edge computing means processing data close to where it is created, like on local devices, instead of sending it to a faraway server. The authors looked at many past studies to understand the main benefits and problems related to security in edge computing. They used a simple method to understand the types of threats and issues that can happen. Their research shows that since data is handled closer to users, it can be more at risk of attacks. So, strong security rules and systems are needed. They also talk about tools that can detect threats early. This helps keep user data safe and the system running smoothly.

Deng, Shancang et al. [23] explore how edge computing is built and how to keep it secure. Edge computing processes data close to the source, like phones or sensors, instead of sending it to distant servers. The authors reviewed many earlier studies to understand how edge computing works and what problems it might face. They focused on the structure (architecture) of the system and the security needed to protect it. Their research shows that edge computing helps make systems faster and more responsive. But it also brings new security challenges because data is handled in many different places.

The study highlights the need for strong protection methods and smart system design. This helps ensure safety and better performance.

Zhang, Qi et al. [24] explore the idea of edge computing and how it is used today. Edge computing means processing data closer to where it is created, like in local devices, instead of faraway servers. The authors reviewed many past studies to understand the latest progress and ongoing problems in this area. They used a method that focuses on understanding ideas rather than numbers. Their study explains that edge computing helps improve speed and reduces delay. However, it also faces challenges like keeping data safe, handling large systems, and managing limited resources. The authors say that more research is needed to solve these problems. This will help make edge computing more effective and secure in the future.

Xu, Xiong et al. [25] focus on how edge computing can help process financial data quickly and accurately. In finance, making fast decisions is very important, and delays (latency) can cause problems. The authors suggest using edge computing, which means handling data close to where it is created, to reduce these delays. They introduce a special system called a latency-aware edge computing framework, designed to analyze financial data in real time. This helps financial companies make quicker and better decisions. Their study shows that reducing latency improves performance and accuracy. It also makes financial systems more responsive and reliable. This is very useful for tasks like stock trading and risk management.

Xu, Zongpeng et al. [26] explore how big data analytics is used in financial systems. Big data analytics means analyzing large amounts of data quickly to find useful information. The authors explain that in finance, this helps detect fraud and monitor transactions in real time. It also plays a key role in high-frequency trading, where computers use live market data to make fast trading decisions. Their study shows that using big data makes financial systems smarter and faster. It helps companies respond quickly to risks and opportunities. The authors highlight the many benefits of this technology, including better accuracy, speed, and security in financial operations.

Lin, Yu-Chee et al. [27] explore how IoT (Internet of Things) and edge computing can help make quick decisions in financial markets. IoT refers to smart devices that collect and share data, while edge computing processes that data close to where it's created. The authors suggest a special system that combines both technologies to support real-time decision-making. Their study shows that using this setup can help financial systems respond faster to market changes. This is important for tasks like

trading, risk analysis, and monitoring. By processing data locally, the system reduces delay and improves speed. The authors highlight that this approach makes financial operations more efficient and smarter.

Li, Yong et al. [28] focus on how edge computing can improve algorithmic trading in financial markets. Algorithmic trading uses computer programs to make fast trading decisions based on real-time data. The authors explain that edge computing helps by processing this data quickly and close to where it is collected. This reduces delays and increases the accuracy of trading decisions. Their study highlights the benefits of using edge computing for real-time analytics, such as faster response times and better performance. This allows financial systems to act quickly during market changes. The authors show that edge computing makes trading systems more efficient, reliable, and competitive.

Wang, Pie et al. [29] study how edge computing can help manage risks in financial services in real time. They reviewed many previous studies to find out the main advantages and challenges of using edge computing for risk management. Edge computing means processing data close to where it is created, which helps make things faster. The authors propose a special system that uses edge computing to improve how financial risks are monitored and handled instantly. Their study shows that this approach leads to faster data processing and more accurate decisions. This helps financial companies respond quickly to potential problems and reduce losses. Overall, edge computing makes risk management more effective and reliable.

Zhao,Liang et al. [30] explore how edge computing can improve real-time economic forecasting. Economic forecasting means predicting how the economy will perform in the near future. The authors reviewed many earlier studies to understand the main benefits and challenges of using edge computing for this purpose. Edge computing processes data close to where it is collected, which helps speed up analysis. The authors propose a system that uses edge computing to make faster and more accurate economic predictions. Their study shows that this approach helps decision-makers respond quickly to changes in the economy. Overall, edge computing makes economic forecasting more efficient and reliable.

Chen,Lian et al. [31] explore how machine learning can be used at the edge to make real-time financial decisions. Machine learning means teaching computers to learn from data and make predictions or decisions. The authors reviewed many earlier studies to understand the benefits and challenges of

running machine learning on edge devices, which process data close to where it is created. They propose a system that uses edge machine learning to help financial systems make faster and more accurate decisions. Their study shows that this approach improves the speed of data processing and the quality of predictions. Overall, edge machine learning helps make financial decisions smarter and quicker.

Chen, Wei et al. [32] study how distributed edge analytics can be used to process economic data streams in real time. Distributed edge analytics means analyzing data across many edge devices instead of one central system. The authors reviewed past research to understand the main benefits and challenges of this approach. They propose a system that uses distributed edge analytics to handle economic data quickly and efficiently. Their study shows that this method allows faster data processing and provides more accurate insights. This helps decision-makers understand the economy better and respond faster to changes. Overall, distributed edge analytics improves real-time economic data analysis.

Chen, Xiang et al. [33] explore how to process financial data with very low delay in edge computing environments. Low-latency processing means handling data quickly so decisions can be made almost instantly. The authors reviewed many previous studies to find the main benefits and challenges of this fast data processing. They propose a system designed to process financial data with minimal delay using edge computing. Their study shows that low-latency processing helps improve financial decision-making by allowing faster and more accurate responses. Overall, this approach makes financial systems quicker and more efficient.

Guo, Xiaohu et al. [34] study how to achieve low-latency communication for edge computing using 5G networks. Low-latency communication means sending and receiving data very quickly, with minimal delay. The authors reviewed many earlier studies to understand the main advantages and challenges of fast communication in 5G networks. They propose a system designed to support low-latency communication for better edge computing performance. Their study shows that faster communication helps edge computing work more efficiently and respond quickly. Overall, low-latency communication improves the speed and reliability of edge computing in 5G networks.

Wang, Jian et al. [35] explore ultra-reliable and low-latency edge computing networks for industrial IoT (Internet of Things). Ultra-reliable low-latency networks are designed to send data very quickly and with almost no errors. The authors reviewed many earlier studies to find the key benefits and challenges of these networks. They propose a system that uses ultra-reliable, low-delay communication

to improve edge computing in industrial settings. Their study shows that these networks help make industrial IoT applications faster, safer, and more dependable. Overall, this approach improves how machines and devices work together in factories and industries.

Cao, Zhitao et al. [36] study how to reduce latency, or delay, in real-time edge computing. Latency minimization means making data processing as fast as possible. The authors reviewed many earlier studies to understand the benefits and challenges of lowering latency in edge computing. They propose a system designed to minimize delay and improve speed in real-time edge applications. Their study shows that reducing latency helps make edge computing faster and more effective. This leads to better performance in applications that need quick responses, such as smart devices and real-time monitoring. Overall, latency minimization improves the speed and reliability of edge computing.

Zhang, Kai et al. [37] explore network slicing as a way to improve low-latency edge computing. Network slicing means dividing a physical network into multiple virtual parts, each tailored for different needs. The authors reviewed many studies to understand the benefits and challenges of using network slicing for faster edge computing. They propose a system that uses network slicing to help edge computing work with very low delays. Their study shows that network slicing improves the speed and efficiency of edge computing. Overall, this approach helps deliver better performance for applications that need quick and reliable data processing.

Liu, Jie et al. [38] study how to process data quickly with low delay at the edge for financial applications. Edge computing means handling data close to where it is created instead of sending it far away. The authors reviewed many earlier studies to find the main benefits and challenges of using edge computing in finance. They propose a system designed to process financial data with low latency at the edge. Their study shows that this approach helps improve financial decision-making by making data processing faster and more accurate. Overall, edge computing helps financial systems work more efficiently and respond quickly to market changes.

Zhou, Ziyuan et al. [39] explore edge intelligence, which combines edge computing and artificial intelligence (AI). Edge intelligence means using AI to analyze data right where it is created, on local devices, instead of sending it to distant servers. The authors reviewed many previous studies to understand the main benefits and challenges of this approach. They propose a system that uses edge intelligence to help make faster and smarter real-time decisions. Their study shows that edge

intelligence improves how quickly and accurately decisions are made. Overall, this technology helps systems become more efficient and responsive in real-world applications.

Chen, Min et al. [40] study how artificial intelligence (AI) can be used at the edge to support real-time decision-making in economic systems. Edge AI means running AI processes directly on local devices near where data is generated, instead of sending data to distant servers. The authors reviewed many earlier studies to understand the benefits and challenges of using AI at the edge. They propose a system that applies edge AI to help economic systems make faster and better decisions. Their study shows that edge AI improves the speed and accuracy of decision-making. Overall, this approach makes economic systems smarter, quicker, and more efficient.

Wang, Shuai et al. [41] study how federated learning and edge analytics can be used in financial markets. Federated learning is a way for multiple devices to learn from data together without sharing the data itself, which helps protect privacy. Edge analytics means processing data close to where it is created. The authors reviewed many earlier studies to understand the benefits and challenges of combining these two technologies. They propose a system that uses federated learning and edge analytics to improve financial market analysis. Their study shows that this approach helps keep data secure while making faster and smarter decisions. Overall, it improves efficiency and privacy in financial data processing.

Ren, Jie et al. [42] explore how edge computing can improve real-time financial forecasting. Financial forecasting means predicting future market trends and prices. The study shows that processing data quickly and close to its source using edge computing can make forecasts more accurate and faster. The authors propose a system that uses edge computing to handle financial data in real time. Their study highlights how this approach helps financial experts make better decisions based on timely information. Overall, edge computing makes financial forecasting more efficient and reliable.

Li, Kezhi et al. [43] present a comprehensive study on real-time edge AI for stock market predictions, aiming to enhance both speed and accuracy in financial forecasting. The authors introduce a novel edge AI framework that processes data locally at the edge, significantly reducing latency compared to traditional cloud-based methods. This approach allows for real-time decision-making, which is crucial in the volatile and fast-paced stock market environment. By minimizing data transmission delays and leveraging on-device intelligence, the model improves response times and reliability. The framework

integrates machine learning models capable of analyzing stock trends and market signals directly on edge devices. The study also demonstrates improved prediction accuracy and system scalability, making it suitable for large-scale deployments. Overall, the research shows how edge AI transforms financial analytics, offering timely insights and competitive advantages in trading strategies.

He, Zhengbing et al. [44] explore the use of online machine learning at the edge for supporting economic decision-making in dynamic environments. The study introduces an edge-based online learning framework that continuously updates models in real-time as new data becomes available. This approach enables rapid adaptation to changing economic conditions, enhancing the relevance and accuracy of decision support. By processing data locally at the edge, the framework reduces latency and ensures timely insights. The authors highlight the system's ability to function efficiently in resource-constrained edge environments. The framework supports various economic scenarios, including pricing strategies and market forecasting. The study concludes that edge-based online learning significantly improves responsiveness and decision quality in economic systems.

Cisco [45] provides a detailed report on its edge computing solutions designed for real-time analytics across various industries. The report emphasizes the advantages of using Cisco's technologies, such as enhanced speed, stronger security, and greater operational efficiency. Cisco's edge solutions support real-time data processing close to the data source, reducing latency and improving responsiveness. These solutions integrate with Internet of Things (IoT) devices, artificial intelligence applications, and advanced data analytics platforms. Cisco offers scalable architectures that can be deployed across edge locations, from factories to retail environments. The systems are built with robust security features to protect data and ensure compliance with industry standards. Overall, Cisco's edge computing solutions aim to empower organizations with faster insights and more efficient data-driven decision-making.

IBM [46] presents a report on the role of edge analytics in enabling real-time business decision-making across various sectors. The report outlines how edge analytics enhances speed, agility, and competitiveness by processing data directly at the source. IBM's edge analytics solutions allow businesses to analyze data in real-time, reducing dependency on cloud processing and minimizing latency. These solutions help organizations respond quickly to changing conditions and customer needs. IBM offers a suite of tools that integrate with AI, IoT, and cloud systems to support smart decision-making at the edge. The platform is designed for scalability and supports deployment in diverse environments such as manufacturing, retail, and transportation. By leveraging edge analytics,

businesses can improve operational efficiency and drive better outcomes through timely and informed decisions.

Gartner [47] provides a comprehensive market guide for edge computing, offering an overview of current trends, key challenges, and emerging opportunities in the field. The report outlines how edge computing is becoming essential for supporting real-time processing, especially in data-intensive and latency-sensitive applications. It highlights the growing adoption of edge technologies across industries such as manufacturing, healthcare, and retail. Gartner identifies major challenges including security, management complexity, and integration with existing IT infrastructure. The guide offers strategic recommendations for organizations planning to implement edge computing, focusing on aligning solutions with business objectives. It also evaluates different edge architectures and vendor offerings to help businesses make informed decisions. Overall, the report serves as a valuable resource for navigating the evolving edge computing landscape.

McKinsey [48] presents a report on the transformative role of edge computing in the financial services sector, focusing on its ability to unlock real-time decision-making. The report emphasizes how edge computing enhances processing speed, operational efficiency, and the accuracy of financial decisions. By enabling data processing closer to the source, financial institutions can reduce latency and respond swiftly to market changes and customer needs. McKinsey outlines use cases such as fraud detection, algorithmic trading, and personalized banking services powered by edge technologies. The report also identifies challenges like data security, infrastructure integration, and regulatory compliance. It provides strategic recommendations for financial institutions aiming to implement edge computing, including investing in scalable platforms and cross-functional teams. Overall, the report underscores edge computing as a key driver for innovation and competitiveness in financial services.

Accenture [49] explores the role of edge intelligence in enabling real-time economic decision-making across industries. The report highlights how edge intelligence improves the speed, accuracy, and effectiveness of decisions by processing data locally at the edge. Accenture's edge solutions combine advanced analytics, artificial intelligence, and IoT capabilities to deliver timely insights directly where data is generated. This allows businesses to act immediately on critical information without relying on centralized cloud processing. The report outlines key benefits such as enhanced responsiveness, reduced latency, and better resource optimization. It also presents real-world applications in sectors like manufacturing, logistics, and energy, where fast, data-driven decisions are crucial. Accenture offers

strategic guidance and scalable platforms for organizations looking to adopt edge intelligence and improve operational outcomes.

Yu, Weisong et al. [50] authored a comprehensive book on edge computing that covers a wide range of topics including edge computing models, technologies, and applications. The book provides detailed explanations of various edge computing architectures, highlighting how data processing and storage can be efficiently distributed closer to data sources. It also delves into edge AI, exploring how artificial intelligence techniques can be integrated with edge devices to enable real-time analytics and decision-making. Additionally, the book discusses edge analytics and its role in processing large volumes of data at the edge to reduce latency and bandwidth usage. Designed for researchers, practitioners, and students, the book offers both theoretical foundations and practical insights, making it a valuable resource for those interested in advancing knowledge and applications in the field of edge computing.

Satyanarayanan, Mahadev et al. [51] authored a book titled "Edge Computing: A Primer" that serves as an introductory guide to the field of edge computing. The book explains the fundamental concepts, including the definition and evolution of edge computing over time. It covers the historical development of edge computing technologies and highlights their growing importance in modern computing environments. The authors discuss various applications of edge computing across industries such as healthcare, manufacturing, and smart cities. The book is designed to be accessible for researchers, practitioners, and students, providing foundational knowledge as well as practical insights. Overall, it aims to equip readers with a clear understanding of edge computing's role and potential in enabling distributed and real-time data processing.

Wang, Jian et al. [52] Real-time systems and edge computing. This book explores the close relationship between real-time systems and edge computing, focusing on how edge computing architectures support time-sensitive applications. It covers essential concepts such as system design, latency reduction, and resource management at the edge. The authors discuss various real-time use cases where edge computing improves responsiveness and reliability. Topics include edge device integration, communication protocols, and scheduling algorithms for real-time tasks. The book is intended for researchers and practitioners working at the intersection of real-time computing and edge technologies. It also addresses challenges in deploying scalable and efficient edge systems for real-time environments. Overall, the book provides practical insights and emerging field.

Chen, Min et al. [53] AI and edge computing for finance

This book investigates how artificial intelligence combined with edge computing is transforming financial services. It emphasizes the role of edge AI in enabling real-time financial analytics, which helps institutions make faster and more accurate decisions. The authors cover use cases like fraud detection, risk assessment, and algorithmic trading powered by edge devices. The book also explores technical challenges, including data privacy, model deployment, and computational constraints at the edge. It targets researchers and practitioners interested in applying AI and edge computing to finance. The discussion includes frameworks and tools for integrating AI models with edge architectures. The book aims to bridge the gap between AI innovation and financial technology applications.

Li, X et al. [54] Financial technology and edge computing. This book focuses on the intersection of financial technology (fintech) and edge computing, explaining how edge computing supports the growing demands of fintech applications. It discusses how edge computing enables low-latency processing, real-time data analysis, and enhanced security for financial services. Topics include edge architectures tailored for fintech, digital payments, blockchain, and customer analytics. The authors highlight practical use cases where edge computing improves transaction speeds and fraud detection. The book is designed for researchers and practitioners working on innovative fintech solutions. It also covers challenges such as regulatory compliance and infrastructure integration. Overall, it presents edge computing as a key enabler for next-generation financial technologies.

Satyanarayanan, M et al. [55] Edge analytics in the Internet of Things. This paper explores the application of edge analytics within the Internet of Things (IoT) environment. It focuses on how edge computing architectures can process and analyze data generated by IoT devices locally to reduce latency and bandwidth usage. The authors discuss key real-time analytics use cases in smart homes, healthcare, and industrial IoT. The paper highlights the benefits of edge analytics in providing timely insights and reducing dependency on cloud infrastructure. It also examines technical challenges such as data heterogeneity, security, and system scalability. The work is targeted toward researchers interested in advancing edge analytics technologies. Overall, the paper emphasizes the critical role of edge computing in unlocking the full potential of IoT applications.

Bonomi, F et al. [56] Fog computing and its role in the Internet of Things. This paper introduces fog computing as a distributed computing paradigm that extends traditional cloud computing closer to the edge of the network, near IoT devices. It highlights how fog computing addresses latency, bandwidth,

and security challenges by enabling local processing and storage. The authors discuss the architecture of fog computing, emphasizing its layered design and scalability. The paper illustrates various IoT applications that benefit from fog computing, such as smart cities and industrial automation. It is regarded as one of the foundational works that established fog computing as a key technology for IoT. The authors also explore the synergy between fog and cloud computing. The paper sets the stage for further research and development in edge and fog computing domains.

Deng, Shancang et al. [57] Edge computing security: State of the art and challenges. This comprehensive survey paper focuses on the security aspects of edge computing, reviewing existing threat models and identifying critical security challenges. The authors discuss various attack vectors that target edge environments, including data breaches, unauthorized access, and denial-of-service attacks. They examine defense mechanisms such as encryption, authentication, and secure hardware solutions tailored for edge devices. The paper emphasizes the importance of securing edge computing infrastructures due to their distributed and resource-constrained nature. Future research directions proposed include enhancing privacy, trust management, and intrusion detection in edge environments. The survey provides valuable insights for researchers and practitioners aiming to strengthen edge computing security.

Varghese, Blessen et al. [58] Edge computing: Principles and practice. This paper provides an overview of the fundamental principles and practical aspects of edge computing. It covers the architecture of edge computing systems, discussing how computation and data storage are distributed closer to data sources. The authors explore key applications and use cases where edge computing delivers benefits such as reduced latency, improved privacy, and bandwidth savings. Challenges such as resource management, scalability, and security are also addressed. The paper highlights the transformative potential of edge computing across diverse domains like healthcare, transportation, and manufacturing. It serves as a valuable resource for understanding both theoretical and applied dimensions of edge computing.

Mach, P et al. [59] Mobile edge computing: A survey on architecture and computation offloading. This survey paper examines mobile edge computing, focusing on its architecture and mechanisms for computation offloading. The authors describe how mobile edge computing brings cloud-like capabilities to the edge of mobile networks, improving application performance and user experience. Various offloading strategies are discussed, including partial and full task offloading, to optimize resource usage and reduce latency. The paper covers use cases such as augmented reality, video

streaming, and gaming, which benefit from mobile edge computing. Challenges such as energy efficiency, mobility management, and security are also analyzed. The survey provides a comprehensive understanding of mobile edge computing research and practical deployments.

Munir, S et al. [60] Real-time big data stream processing in IoT: Edge computing and cloudlet technologies. This paper explores the use of edge computing and cloudlet technologies to enable real-time big data stream processing in IoT applications. The authors highlight the challenges of processing large volumes of IoT data efficiently and the benefits of distributing computation close to data sources. Cloudlets are discussed as small-scale data centers that complement edge devices by providing additional computational power nearby. The paper reviews architectures and frameworks designed to support real-time analytics, emphasizing latency reduction and bandwidth optimization. Use cases include smart cities, healthcare monitoring, and industrial IoT. The paper suggests future research directions for improving scalability and resource allocation in these environments.

Hassan, W et al. [61] Real-time applications in edge computing: Architectures and challenges. This paper investigates architectures that support real-time applications within edge computing environments. It outlines how edge computing enables timely data processing critical for applications like autonomous vehicles, smart grids, and healthcare systems. The authors discuss architectural designs that prioritize low latency and high reliability. They identify challenges such as resource constraints, network variability, and security concerns that affect real-time performance. The paper also explores middleware solutions and scheduling algorithms tailored to real-time edge applications. It highlights opportunities for further research to enhance the capabilities and robustness of real-time systems at the edge.

Wang, S et al. [62] Real-time big data processing and analytics on edge computing. This paper focuses on the use of edge computing for real-time big data processing and analytics. It discusses how processing data locally at the edge reduces latency and bandwidth usage compared to centralized cloud approaches. The authors describe architectures that support continuous data ingestion, processing, and analytics near data sources. Key applications covered include smart cities, industrial IoT, and environmental monitoring. The paper analyzes challenges such as data heterogeneity, scalability, and security in edge environments. It also highlights emerging technologies and frameworks that facilitate real-time big data analytics on the edge. The work emphasizes the transformative potential of edge computing in managing large-scale real-time data streams.

Lin, K et al. [63] Latency-aware task offloading and resource allocation for edge computing systems. This paper proposes a latency-aware scheme for task offloading and resource allocation in edge computing systems. The authors aim to minimize processing delays by intelligently deciding which tasks should be offloaded to edge servers versus processed locally. The scheme considers network conditions, resource availability, and task requirements to optimize system performance. Simulation results demonstrate improvements in latency reduction and overall system efficiency. The paper highlights the importance of latency awareness in ensuring quality of service for time-sensitive edge applications. It also discusses potential extensions to support dynamic environments and multi-user scenarios. The approach contributes to the development of more responsive and efficient edge computing systems.

Hu, Y et al. [64] Mobile edge computing: Challenges and future directions. This paper reviews the current challenges faced by mobile edge computing and outlines possible future research directions. Key issues discussed include resource management, mobility support, security, and privacy in mobile edge environments. The authors examine how mobile edge computing can enhance application performance but also highlight limitations such as hardware constraints and complex network management. Emerging trends like integration with 5G networks and AI-driven edge services are discussed as promising areas for future development. The paper serves as a roadmap for researchers and practitioners aiming to address the obstacles and harness the full potential of mobile edge computing.

Yousefpour, A et al. [65] All one needs to know about fog computing and related edge computing paradigms. This paper provides a thorough overview of fog computing and related edge computing paradigms, explaining their definitions, architectures, and differences. The authors highlight the benefits of these paradigms, such as reduced latency, enhanced security, and improved scalability. They discuss the relationships and overlaps between fog computing, mobile edge computing, and other edge paradigms. The paper surveys key applications and use cases, including smart cities, industrial automation, and healthcare. Challenges such as interoperability, management complexity, and security are also analyzed. This comprehensive review serves as a foundational resource for understanding the broader edge computing ecosystem.

Shi, W et al. [66] Edge computing: Vision and challenges. This seminal paper introduces edge computing as a paradigm that brings computation and data storage closer to the network edge, near data sources. It outlines the vision of edge computing in addressing issues like latency, bandwidth constraints, and data privacy. The authors discuss the potential benefits such as improved real-time processing, enhanced user experience, and energy efficiency. Key challenges identified include system heterogeneity, resource management, and security. The paper also sets a research agenda for advancing edge computing technologies. It is widely recognized as one of the foundational works that shaped the field and inspired subsequent research and development efforts.

Mach, P et al. [67] Edge computing for real-time data analytics: A survey. This paper offers a comprehensive survey on edge computing focused on real-time data analytics. It covers various edge computing architectures designed to handle data processing close to data sources. The authors discuss data processing techniques and analytics frameworks optimized for edge environments. Benefits such as reduced latency, improved scalability, and bandwidth efficiency are highlighted. Challenges including resource constraints, security, and data management are examined. The paper also identifies open research issues for advancing real-time analytics at the edge. It serves as a foundational reference for researchers and practitioners in edge analytics.

Chen, M et al. [68] Distributed real-time processing at the edge: Emerging trends and open issues. This paper explores the latest trends in distributed real-time data processing using edge computing. It highlights how edge computing enables decentralized, timely analytics closer to data generation points. The authors discuss key challenges such as synchronization, scalability, and fault tolerance in distributed edge systems. Emerging technologies that support real-time processing at the edge are reviewed. The paper identifies open research problems including resource management, security, and integration with cloud platforms. The benefits of edge computing in enhancing responsiveness and reducing data transmission overheads are emphasized. It provides insights for future research directions in distributed edge processing.

Xie, S et al. [69] Latency minimization in real-time edge computing. This paper proposes a scheme aimed at minimizing latency in real-time edge computing applications. It addresses the critical need for reducing delays to ensure timely processing and response. The authors develop algorithms that optimize task scheduling and resource allocation to achieve low latency. Experimental results

demonstrate improvements in performance and reduced processing time. The paper discusses the importance of latency minimization in applications such as IoT, autonomous systems, and real-time analytics. Challenges like network variability and resource constraints are considered. The work contributes valuable strategies for enhancing edge computing responsiveness.

Dong, Y et al. [70] Real-time algorithmic trading via edge computing systems. This paper investigates the application of edge computing to support real-time algorithmic trading. It discusses how processing trading data closer to the source reduces latency and improves decision-making speed. The authors analyze challenges such as data security, system reliability, and compliance in edge-based trading systems. Benefits like faster trade execution and improved market responsiveness are highlighted. The paper proposes architectural considerations and frameworks for deploying edge computing in financial trading. It emphasizes the potential of edge computing to transform algorithmic trading with low latency and high performance. Future research directions include enhancing scalability and security.

Chen, X et al. [71] Edge-enabled financial fraud detection in real-time. This paper explores how edge computing can enhance real-time financial fraud detection. By processing transaction data at the edge, detection systems can identify fraudulent activities faster and more accurately. The authors discuss system architectures that combine edge computing with machine learning for fraud detection. Benefits include reduced response times and decreased reliance on centralized cloud processing. Challenges such as data privacy, scalability, and false positive reduction are examined. The paper highlights the potential of edge-enabled fraud detection to improve financial security and customer trust. It also outlines directions for future research to refine detection algorithms and system deployment.

Lin, Y et al. [72] Real-time financial market forecasting with edge-based deep learning. This paper presents an approach using edge computing combined with deep learning for real-time financial market forecasting. The system processes data locally at the edge to reduce latency and improve forecasting speed. The authors show how edge-based models enhance accuracy by enabling timely analysis of market trends. The approach supports dynamic and high-frequency market conditions. Benefits include improved decision-making and reduced data transmission costs. Challenges such as model updating and resource limitations are discussed. The paper illustrates the growing role of AI at the edge in financial analytics.

Li, W et al. [73] Edge computing for high-frequency trading: Low latency strategies. This paper examines how edge computing supports low latency strategies in high-frequency trading (HFT). It explains the critical role of minimizing delay to maximize trading profits in HFT environments. The authors discuss architectural designs that deploy computation and data storage near financial exchanges. Benefits include ultra-fast data processing, reduced network jitter, and improved order execution speed. The paper also addresses challenges related to synchronization, data consistency, and system reliability. It highlights the potential of edge computing to revolutionize HFT by delivering superior latency performance. Future work involves integrating AI for predictive trading.

Wang, Z et al. [74] Real-time risk assessment for financial markets using edge analytics. This paper proposes using edge analytics for real-time risk assessment in financial markets. The approach processes data close to the source to rapidly evaluate market risks and exposures. Benefits include faster detection of market anomalies and improved decision accuracy. The authors describe edge computing architectures that support continuous monitoring and risk scoring. Challenges such as data heterogeneity, security, and scalability are discussed. The paper highlights how edge analytics can enhance financial stability and compliance. It also suggests directions for integrating AI and blockchain technologies in risk assessment.

Zhang, Q et al. [75] Distributed edge analytics for real-time credit scoring. This paper explores distributed edge analytics as a method for real-time credit scoring. It emphasizes processing credit-related data at the edge to improve speed and privacy. The authors propose architectures that distribute computation across edge devices to handle large volumes of credit data. Benefits include enhanced scoring accuracy, reduced latency, and better data security. Challenges such as data synchronization, model consistency, and resource management are examined. The paper discusses the potential of edge computing to transform credit evaluation processes in financial institutions. Future work includes improving machine learning models for edge deployment.

He, J et al. [76] Real-time economic decision support via edge computing and AI. This paper investigates the combination of edge computing and AI for real-time economic decision support. The authors highlight how edge-enabled AI systems can analyze economic data faster and more accurately. Benefits include improved decision quality, reduced latency, and enhanced responsiveness to economic changes. The paper proposes architectures integrating edge devices with AI models to

support decision-making processes. Challenges like data privacy, computational constraints, and model interpretability are discussed. The authors suggest that edge-AI systems will play a key role in future economic analytics and policy making.

Li, S et al. [77] Edge computing for real-time market microstructure analysis. This paper proposes an edge computing approach for real-time market microstructure analysis, which studies market behaviors and trading mechanisms. The authors highlight benefits such as reduced latency and enhanced accuracy in analyzing high-frequency trading data. The paper discusses system design considerations that enable processing close to data sources. Challenges such as data volume, processing complexity, and synchronization are examined. The authors emphasize how edge computing can provide timely insights to traders and regulators. Potential future work includes integration with AI for predictive analysis.

Zhao, J et al. [78] Real-time portfolio optimization using edge computing. This paper explores using edge computing for real-time portfolio optimization to dynamically adjust asset allocations. The authors demonstrate how edge processing reduces latency and allows faster response to market changes. Benefits include improved optimization accuracy and reduced reliance on cloud infrastructure. The paper discusses architectures that enable distributed computations on edge devices. Challenges such as computational complexity and data security are addressed. The authors highlight the potential for edge computing to enable personalized and adaptive portfolio management solutions.

Xu, H et al. [79] Edge-enabled real-time financial data stream processing. This paper proposes an edge-enabled framework for real-time processing of financial data streams. It highlights how local processing reduces delays and bandwidth consumption compared to centralized systems. The authors discuss architectures and algorithms optimized for streaming data analytics at the edge. Benefits include faster anomaly detection, enhanced market surveillance, and improved data quality. Challenges such as data integration, security, and resource allocation are considered. The paper illustrates the growing importance of edge computing in handling continuous financial data streams.

Liu, J et al. [80] Real-time machine learning on edge devices for economic forecasting. This paper explores the use of real-time machine learning on edge devices to improve economic forecasting. The authors demonstrate that edge deployment allows faster model updates and predictions with reduced

communication overhead. Benefits include improved forecasting accuracy and timely decision support. The paper discusses challenges such as limited computational resources and model complexity on edge devices. The authors propose lightweight algorithms suitable for edge environments. This work underscores the growing synergy between edge computing and AI in economic analysis.

2.3 RESEARCH GAPS

Scalability Across Diverse IoT Applications: The study primarily focuses on a specific smart city use case. There is a lack of comprehensive analysis regarding the scalability of edge computing solutions across diverse and heterogeneous IoT domains such as healthcare, industrial automation, and agriculture.

Security and Privacy Considerations: The paper gives limited attention to security and privacy challenges that arise when processing sensitive data at the edge. Further research is needed to explore secure data handling and privacy-preserving techniques in edge-enabled IoT systems.

Limited Exploration of Application-Specific Challenges: While general challenges are discussed, there is insufficient analysis of how these challenges vary across specific domains like healthcare, autonomous systems, or industrial IoT. Tailored solutions for each domain remain underexplored.

Energy Efficiency and Resource Management: Although performance improvements are highlighted, there is insufficient discussion on the energy consumption and resource optimization strategies necessary for sustainable edge deployment in resource-constrained IoT environments.

Dynamic Resource Allocation and Load Balancing: The study lacks an in-depth analysis of how dynamic workloads and changing network conditions impact edge performance. Advanced strategies for load balancing and resource allocation in real-time scenarios need to be explored.

2.4 SUMMARY

The literature over the past five years (2020–2024) reflects a growing interest in leveraging edge computing to support real-time economic decision-making across various domains. Researchers have demonstrated that edge computing, by processing data closer to the source, can significantly reduce latency, improve responsiveness, and enable localized, intelligent decisions in time-sensitive economic environments.

Studies have explored its application in fields such as high-frequency trading, smart grids, e-commerce, transportation, smart agriculture, retail, and micro-market management. These

implementations have shown that edge computing enhances decision speed and efficiency, allowing systems to respond dynamically to changes in market conditions, user behaviour, and environmental inputs.

Despite these advancements, research remains fragmented, with solutions often tailored to specific industries. Additionally, several challenges persist, including the lack of unified frameworks, limited AI integration at the edge, data privacy concerns, and scalability in resource-constrained environments.

Overall, the literature underscores the transformative potential of edge computing for real-time economic optimization while highlighting key areas for further exploration and innovation.

CHAPTER 3

EXISTING SYSTEM

3.1 Centralized Financial Decision-Making

In traditional financial institutions, decisions such as loan approvals, fraud detection, and risk assessments are often made through centralized systems. These systems rely on large data centers where information from various branches is sent for processing. Employees manually analyze spreadsheets, financial documents, and past records to arrive at outcomes. This process is heavily time-consuming and prone to human bias or oversight.

Due to centralization, there's often a significant delay between data collection and decision execution. For example, fraud alerts may take minutes or even hours to be confirmed, during which malicious transactions might go unnoticed. Moreover, decisions are frequently based on static rule-based systems, which cannot adapt quickly to changing economic conditions.

This lack of automation and real-time processing leads to inefficiencies in responding to market volatility. In addition, central systems are highly vulnerable to downtimes or cyber-attacks. The manual validation process also adds a layer of inconsistency, especially in high-volume environments, where scaling decisions quickly becomes nearly impossible without automation.

3.2 Traditional Retail Management

Retail stores historically operate using manual inventory checks, paper-based logs, and in-store observations to make decisions regarding stock restocking, promotions, and pricing. Store managers rely on past experiences and limited historical sales data to predict customer behavior and manage inventory manually. These decisions are often reactive rather than proactive.

For instance, if a product is out of stock, restocking decisions may be delayed until the shortage is noticed during routine checks. This results in lost sales and customer dissatisfaction. Promotions or pricing strategies are also implemented with little to no analytical backing, often based on guesswork or general seasonal trends.

Without real-time insights, retailers struggle to manage perishable goods, respond to sudden demand surges, or predict slow-moving inventory. The lack of automation means that vast amounts of data from point-of-sale systems, customer preferences, and inventory levels are underutilized or entirely unused, leading to inefficiencies in operations and revenue loss.

3.3 Municipal Economic Planning in Smart Cities

In many cities, economic planning and resource allocation still rely on manual systems such as surveys, public meetings, and delayed reporting from municipal departments. Budget decisions, resource deployment (like water, electricity), and infrastructure planning are often based on annual or quarterly reviews rather than continuous real-time data.

Officials gather inputs from various departments, aggregate them manually, and make strategic decisions. This approach lacks immediacy and does not reflect real-time needs, especially in rapidly changing urban environments. For instance, traffic management decisions are based on historical congestion data rather than live sensor inputs, leading to ineffective or outdated strategies.

The absence of automated prediction tools results in slow responses to emergencies, inefficient use of public funds, and delayed infrastructure improvements. Decision-making processes are not only slower but also more susceptible to subjective biases and political influence, which compromises transparency and effectiveness in governance.

3.4 Drawbacks of Manual Systems

- **High Latency:** Manual processes introduce significant delays between data collection and action, making real-time responsiveness impossible.
- **Limited Scalability:** As data volume grows, manual systems fail to scale efficiently, leading to bottlenecks in decision-making.
- **Error-Prone:** Human involvement increases the likelihood of inconsistencies, inaccuracies, and oversight in analysis and decision execution.
- **Data Underutilization:** Vast amounts of collected data remain unprocessed or unused, wasting potential insights.
- **Lack of Adaptability:** Manual systems cannot dynamically adjust to real-time changes in data or economic conditions, reducing effectiveness in fast-paced environments.

CHAPTER 4

PROPOSED METHODOLOGY

4.1 System Architecture

The proposed system is designed to empower real-time economic decision-making through the deployment of a scalable, low-latency, and intelligent edge computing pipeline. The architecture focuses on streamlining the entire machine learning lifecycle — from data ingestion to model deployment — in a manner suitable for edge-based inference. The system starts with structured data collection, followed by automated preprocessing, exploratory analysis, and model training using ensemble algorithms. It features a dual-model approach where Extra Trees is used as a comparative baseline, and CatBoost is proposed as the primary predictive model for its superior handling of categorical data and robustness. The system supports real-time prediction on unseen test data using pre-trained models and an exported scaler, making it suitable for deployment on edge devices with minimal resources.

Step-1: Data Loading

- The system initiates by loading the dataset using a defined utility function `load dataset()`, which reads .csv input data for both training and testing.
- This includes structured features like transaction volume, system throughput, financial metric, and the label decision outcome.

Step-2: Data Preprocessing

- The function `preprocess data()` performs essential data cleaning by dropping missing values and encoding the target label: 'Effective' $\rightarrow$ 1, 'Not Effective' $\rightarrow$ 0.
- It also scales all numeric features using Standard Scaler to ensure that the models perform optimally and to avoid bias due to scale differences.
- The scaler is saved using job lib for consistent transformation during inference on test data.

Step-3: Exploratory Data Analysis (EDA)

- The method performedata() generates five critical visualizations: correlation heatmap, outcome distribution, transaction volume analysis, scatter plot for throughput vs financials, and a histogram for resource usage.
- These plots help in feature interpretation and identification of dominant patterns or anomalies that influence the decision outcomes.

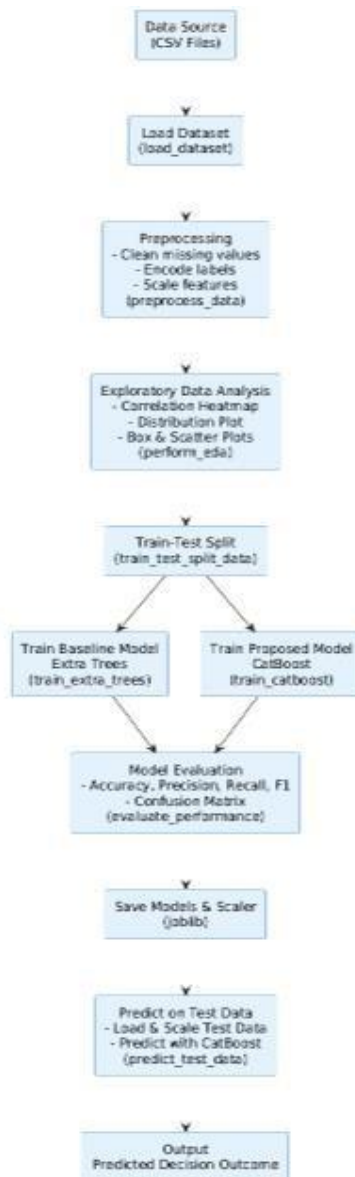


Fig. 4.1: Architecture for Real Time Economic Decision via Edge Computing.

Step-4: Data Splitting

- Using `train test split data()`, the preprocessed dataset is divided into training and test sets with an 80:20 ratio.
- This ensures unbiased model evaluation and reproducibility through fixed random state.

Step-5: Model Training – Baseline (Extra Trees Classifier)

- The function `train extra trees()` is used to train the baseline model using the Extra Trees algorithm.
- This model is chosen for its simplicity, feature ranking capabilities, and quick training time.
- If the model is already trained and saved, it is loaded instead of retraining.

Step-6: Model Training – Proposed (Cat Boost Classifier)

- The function `train catboost()` trains the Cat Boost model with optimized hyperparameters such as 100 iterations, learning rate of 0.1, and depth of 6.
- Cat Boost excels in handling categorical features natively and performs well on small to medium datasets without extensive preprocessing.
- Like the Extra Trees model, the trained Cat Boost model is stored for future use.

Step-7: Model Evaluation

- For both models, predictions are made on the test data, and the results are passed to `evaluate performance()`.
- Performance is assessed using Accuracy, Precision, Recall, F1 Score, and confusion matrix visualization.
- This evaluation ensures that Cat Boost's edge in performance is quantifiably proven against the baseline.

Step-8: Prediction on Unseen Test Data

- The function `predict test data()` accepts a test file path and a model name (e.g., 'Cat Boost'), loads the test dataset, scales it using the saved scaler, and generates predictions.
- The predictions are mapped back to the original labels ('Effective' or 'Not Effective') and appended to the dataset for result export or dashboard integration.

4.2 Data Preprocessing

Data preprocessing is a fundamental step in transforming raw economic decision-making data into a structured format suitable for machine learning models. In this project, preprocessing ensures that the dataset used for training and prediction is clean, standardized, and machine-readable. The process begins by addressing missing values, followed by encoding categorical labels and scaling numeric features. These operations eliminate inconsistencies, reduce noise, and optimize model performance by ensuring that all inputs are on a comparable scale. By executing these steps methodically, the system ensures robust and accurate predictions of decision outcomes.

Step-1: Handling Missing Values

- The dataset is examined for any rows containing missing or null values.
- To maintain data consistency and quality, all records with missing values are dropped from the dataset. This prevents erroneous input from degrading the model's accuracy.

Step-2: Encoding the Target Variable

- For training datasets, the target column decision outcome is categorical and includes two labels: 'Effective' and 'Not Effective'.
- This variable is converted to numerical format using label encoding, where 'Effective' is mapped to 1 and 'Not Effective' to 0. This enables compatibility with machine learning classifiers that require numerical output.

Step-3: Separating Features and Target

- The dataset is divided into features (X) and target (y).
- Features include all columns except the decision outcome, and the target comprises only the encoded outcome. This distinction prepares the data for supervised learning models that predict the target based on feature input.

Step-4: Feature Scaling

- All numerical features are standardized using Standard Scaler, which transforms them to have a mean of 0 and standard deviation of 1.
- This step is critical to eliminate feature dominance due to varying scales, thereby ensuring that all input variables contribute proportionately during training.

Step-5: Saving Preprocessing Objects

- The scaler object used during training is saved for later reuse.
- This guarantees that test datasets are transformed using the exact same scaling parameters, maintaining consistency between training and prediction phases.

4.3 EXPLORATION DATA ANALYSIS

Exploratory Data Analysis (EDA) is a foundational step in understanding the structure, trends, and relationships within a dataset before proceeding with machine learning. In this project, EDA was employed to derive insights into the factors influencing decision outcomes in economic management. Through the use of multiple visualizations, the dataset was analyzed from various perspectives to identify meaningful patterns, correlations, and distributions that are essential for building effective predictive models.

Step-1: Correlation Heatmap

The correlation heatmap is used to quantify the linear relationships between numerical features in the dataset. By calculating the Pearson correlation coefficient, a matrix is generated to show how each variable relates to others. This matrix is then visualized using a heatmap, where colors represent the strength and direction of correlations—ranging from -1 (strong negative) to +1 (strong positive). Features with high

correlations can either be valuable predictors or redundant if they correlate strongly with each other. This visualization is critical for feature selection, multicollinearity detection, and understanding the potential impact of each variable on the target outcome (decision outcome).

Step-2: Distribution of Decision Outcome

This step involves plotting the frequency of each category in the target variable decision outcome, which has two classes: Effective and Not Effective. Using a count plot, the number of instances for each class is displayed, giving immediate insight into class distribution. A balanced dataset is ideal for classification tasks, while a highly imbalanced dataset may bias the model toward the majority class. In this project, the distribution visualization helps to assess whether any resampling or class weighting strategies are required during model training.

Step-3: Box Plot of Transaction Volume by Decision Outcome

The box plot is used to examine the spread and central tendency of transaction volume across different decision outcomes. It highlights the median, quartiles, and any potential outliers for each class. This comparison allows us to understand whether high or low transaction volumes are associated with more effective decisions. A noticeable shift in medians or the presence of outliers in one class may indicate that transaction volume plays a significant role in the effectiveness of decisions. This insight is vital for feature importance evaluation and model interpretability.

Step-4: Scatter Plot of Financial Metric vs System Throughput

This step involves plotting two continuous features—financial metric and system throughput—to observe their interaction and how they influence the decision outcome. Data points are color-coded based on their respective outcomes to visually separate the two classes. This visualization is helpful in identifying clusters, trends, or boundaries between effective and non-effective decisions. If distinct groupings or separations are visible, it suggests that these features are strong candidates for classification. This step not only enhances understanding of feature behavior but also provides intuition about how the model might learn from these patterns.

Step-5: Histogram of Resource Utilization

The histogram provides a distribution plot of the resource utilization feature, offering insight into how frequently different utilization levels occur. It also incorporates a Kernel Density Estimate (KDE) curve to depict the underlying probability distribution. This visualization helps detect skewness, peaks, and possible multimodal distributions, all of which influence the choice of preprocessing and modeling techniques. Understanding the typical and extreme values in resource usage helps in managing system-level constraints and supports model robustness by identifying rare but critical operational scenarios.

4.4 Dataset Splitting

Train-test splitting is a fundamental step in the machine learning workflow, where the original dataset is divided into two subsets: one used for training the model and the other for evaluating its performance. This separation helps to test how well the model generalizes to unseen data, which is crucial for avoiding overfitting. In this project, the `train test split()` function from Scikit-learn is used to perform this task.

Step-1: Define Input Features and Target Variable

Before splitting the data, the dataset is separated into input features (independent variables) and the target variable (decision outcome). The features include all relevant numerical or categorical attributes that the model will use to make predictions, such as transaction volume, system throughput, and resource utilization. The target variable is the outcome the model aims to predict—whether a decision is Effective or Not Effective.

Step-2: Specify Test Size and Random State

The train-test split is configured to allocate 80% of the data for training and 20% for testing. This ratio balances the need for sufficient data to train the model while preserving enough examples to evaluate performance accurately. A random state value is set to ensure reproducibility of results. This means the same split will occur each time the code is executed, which is essential for consistency during development and testing.

Step-3: Generate Training and Testing Sets

Using the specified parameters, the dataset is split into four parts: X train, X test, y train, and y test. The training set (X train, y train) is used to fit the machine learning models, while the testing set (X test, y test) is kept aside to assess the models' predictive accuracy and generalization capability. This clear separation ensures that the model is not evaluated on data it has already seen during training.

Step-4: Ensure Representative Sampling

The train test split function ensures that the sampling of training and testing data is random and representative of the overall dataset distribution. This is especially important when dealing with classification problems, where maintaining a balanced proportion of target classes in both sets is critical to avoid biased evaluation. In cases of class imbalance, additional strategies such as stratified sampling may be employed, although not explicitly shown in the provided code.

4.5 Model Building and Training

4.5(a) Extra Trees Classifier

Step 1: Input Data Preparation

The process begins with the input of the training dataset, where X train consists of the features and y train contains the corresponding decision outcomes. This clean and scaled data is crucial for building a robust model.

Step 2: Model Initialization

The Extra Trees Classifier, an ensemble-based learning algorithm, is initialized with defined parameters such as the number of estimators (trees) and the random seed. This initialization sets up multiple decision trees that will be trained on different feature subsets.

Step 3: Model Training

The Extra Trees model is trained using X_{train} and y_{train} . During training, it builds a forest of unpruned decision trees, where each tree is trained on the full dataset but splits nodes using random thresholds. This increases variance and reduces model bias.

Step 4: Prediction Phase

Once training is completed, the model receives X_{test} as input to predict outcomes. Each tree in the ensemble votes on the classification, and the final output y_{pred} is determined through majority voting.

Step 5: Model Evaluation

The predicted outcomes y_{pred} are compared with the true labels y_{test} . Performance metrics like accuracy, precision, recall, and F1-score are computed to assess how well the model performs on unseen data.

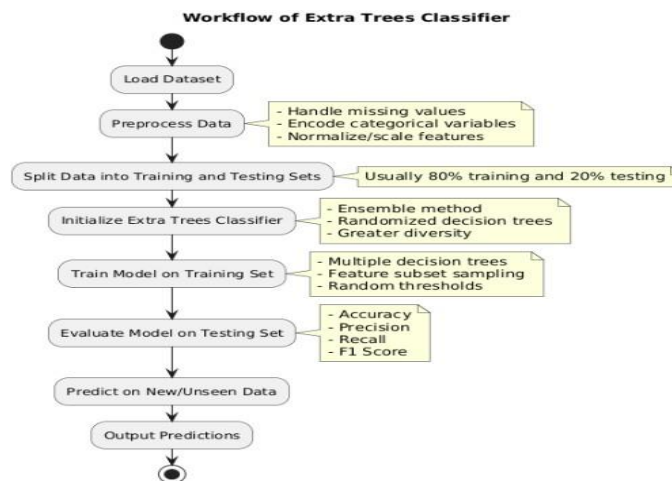


Fig. 4.5(a): Block diagram of working flow of Extra Trees Classifier.

4.5(b) Drawbacks of Extra Trees Classifier

- **Instability on Smaller Datasets**

Random threshold selection in Extra Trees can introduce instability, especially when the dataset is small. Since the splits are not optimized like in decision trees or random forests, the results can

fluctuate significantly based on random variations in the data, leading to inconsistent performance across runs.

- **Higher Model Variance**

Extra Trees increases randomness by selecting thresholds at random rather than using optimal split points. While this helps reduce overfitting in some cases, it also raises the overall variance of the model, making it less stable than more deterministic models like Random Forests.

- **Poor Interpretability**

As the number of trees in the ensemble grows, understanding the contribution of individual features or the logic behind specific predictions becomes difficult. This compromises model transparency, making it challenging for stakeholders to interpret or trust the model's decisions.

- **Noise Sensitivity**

Due to the algorithm's aggressive and non-optimal splitting strategy, Extra Trees can become highly sensitive to noise. Irrelevant or erroneous data can lead to splits that do not generalize well, reducing the overall robustness of the model on unseen datasets.

- **Resource Intensive**

Building a large number of fully grown unpruned trees with random splits is computationally expensive. It requires significant memory and processing power, particularly on large datasets, which can limit scalability and efficiency in real-time or resource-constrained environments.

4.5(c) Cat Boost Classifier

Step 1: Input Data Preparation

Just like Extra Trees, Cat Boost begins with X_{train} and y_{train} as the training data. The algorithm automatically handles categorical features, removing the need for manual encoding.

Step 2: Model Initialization

Cat Boost is initialized with parameters such as the number of iterations, learning rate, and depth. It is known for efficient handling of categorical data and high performance on small-to-medium datasets.

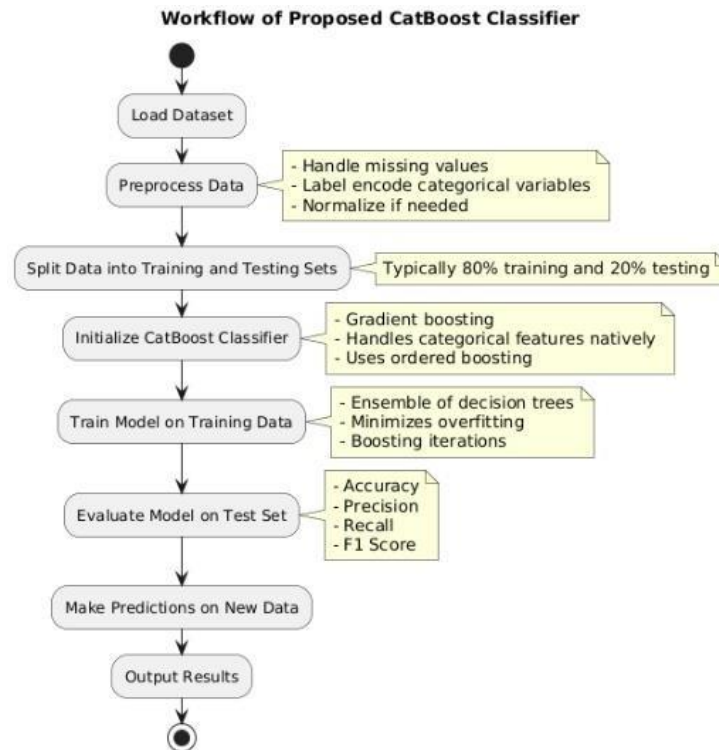


Fig. 4.5(c): Workflow of Proposed Cat Boost Classifier.

Step 3: Model Training

The Cat Boost algorithm uses ordered boosting and symmetric trees to minimize prediction shift. It processes the training data X_{train} and learns patterns against the target variable y_{train} .

Step 4: Prediction Phase

After training, X_{test} is fed into the model. The trained Cat Boost model predicts the outcomes and generates y_{pred} —the predicted labels for the test data.

Step 5: Model Evaluation

The predicted labels y_{pred} are compared with the true labels y_{test} . The model is evaluated using standard classification metrics to determine its effectiveness.

4.5(d) Advantages of Cat Boost Classifier

- **Native Handling of Categorical Variables:** Cat Boost stands out by offering native support for categorical features. Unlike other machine learning algorithms that require manual encoding (like one-hot or label encoding), Cat Boost processes these features internally using advanced techniques like permutation-driven statistics. This eliminates preprocessing overhead, reduces human error, and maintains the natural structure of the data, making the pipeline more streamlined and efficient.
- **Effective Overfitting Control:** The algorithm implements a technique known as ordered boosting, which prevents target leakage and overfitting—a common issue in conventional gradient boosting methods. This approach ensures that the boosting process only uses past data to compute the model's structure at each stage, thereby improving generalization on unseen datasets.
- **Minimal Hyperparameter Tuning:** Cat Boost is designed to deliver high performance even with default parameters. It requires less manual fine-tuning compared to other gradient boosting models like XG Boost or LightGBM. This feature is particularly beneficial for practitioners with limited machine learning expertise or in scenarios where time-to-deployment is critical.
- **Built-in Handling of Missing Values:** Missing values are a challenge in most datasets. CatBoost automatically manages them without requiring any manual imputation or dropping of rows. It learns optimal paths in decision trees by considering missing values as a separate category, thereby preserving the integrity and completeness of the data.
- **Superior Generalization and Prediction Stability:** Thanks to innovations like oblivious trees and permutation-based training, Cat Boost significantly reduces prediction shift and model variance. These features result in more stable and generalized models, which perform reliably across different samples of data and in production environments.

CHAPTER 5

UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language Is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS: The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components. Integrate best practices.

5.1 Class Diagram

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an “is-a” or “has-a” relationship. Each class in the class diagram may be capable of providing certain functionalities. These functionalities provided by the class are termed “methods” of the class. Apart from this, each class may have certain “attributes” that uniquely identify the class.

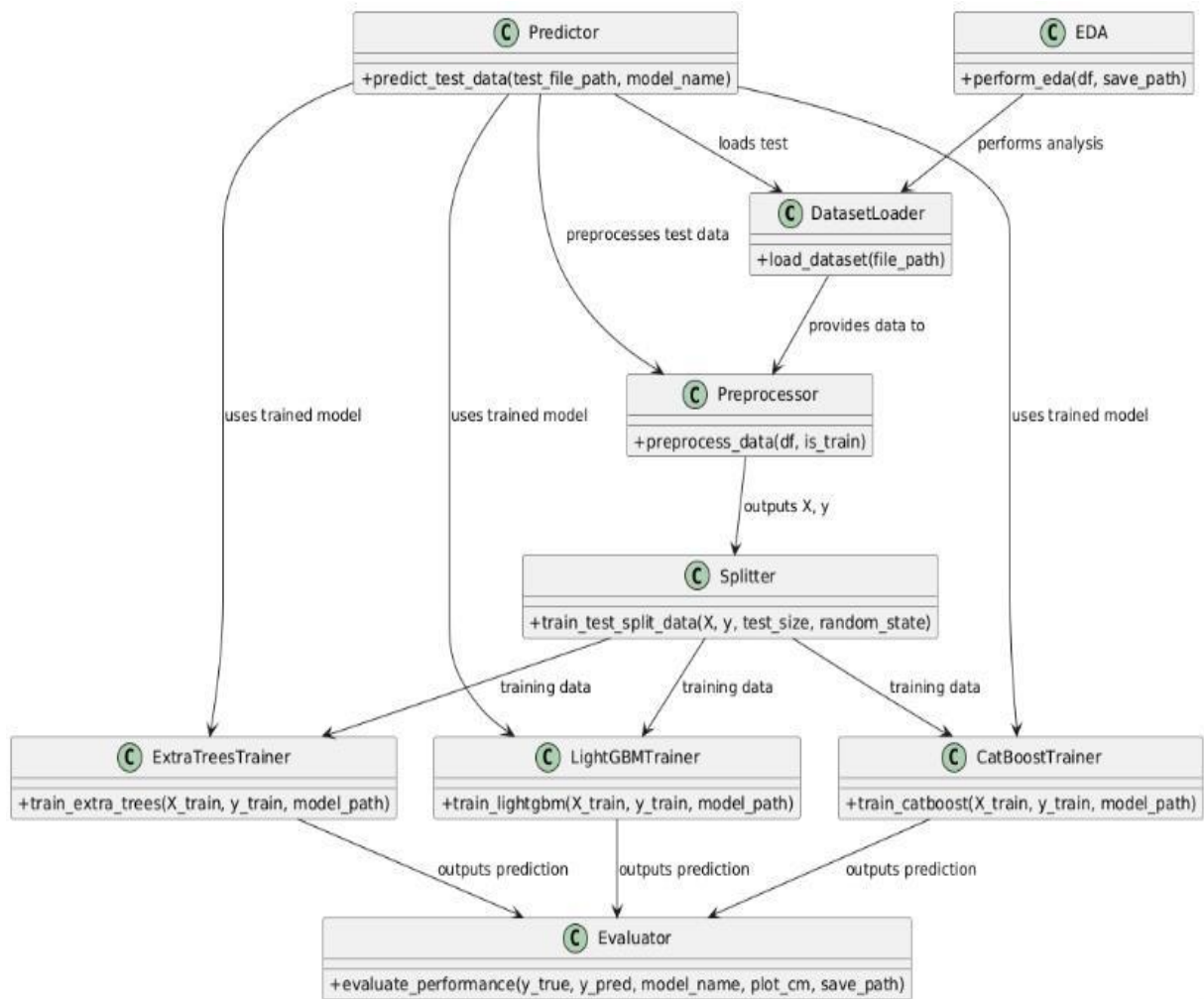


Fig. 5.1: Class Diagram

5.2 Data flow diagram

A Data Flow Diagram (DFD) is a visual representation of the flow of data within a system or process. It is a structured technique that focuses on how data moves through different processes and data stores within an organization or a system. DFDs are commonly used in system analysis and design to understand, document, and communicate data flow and processing.

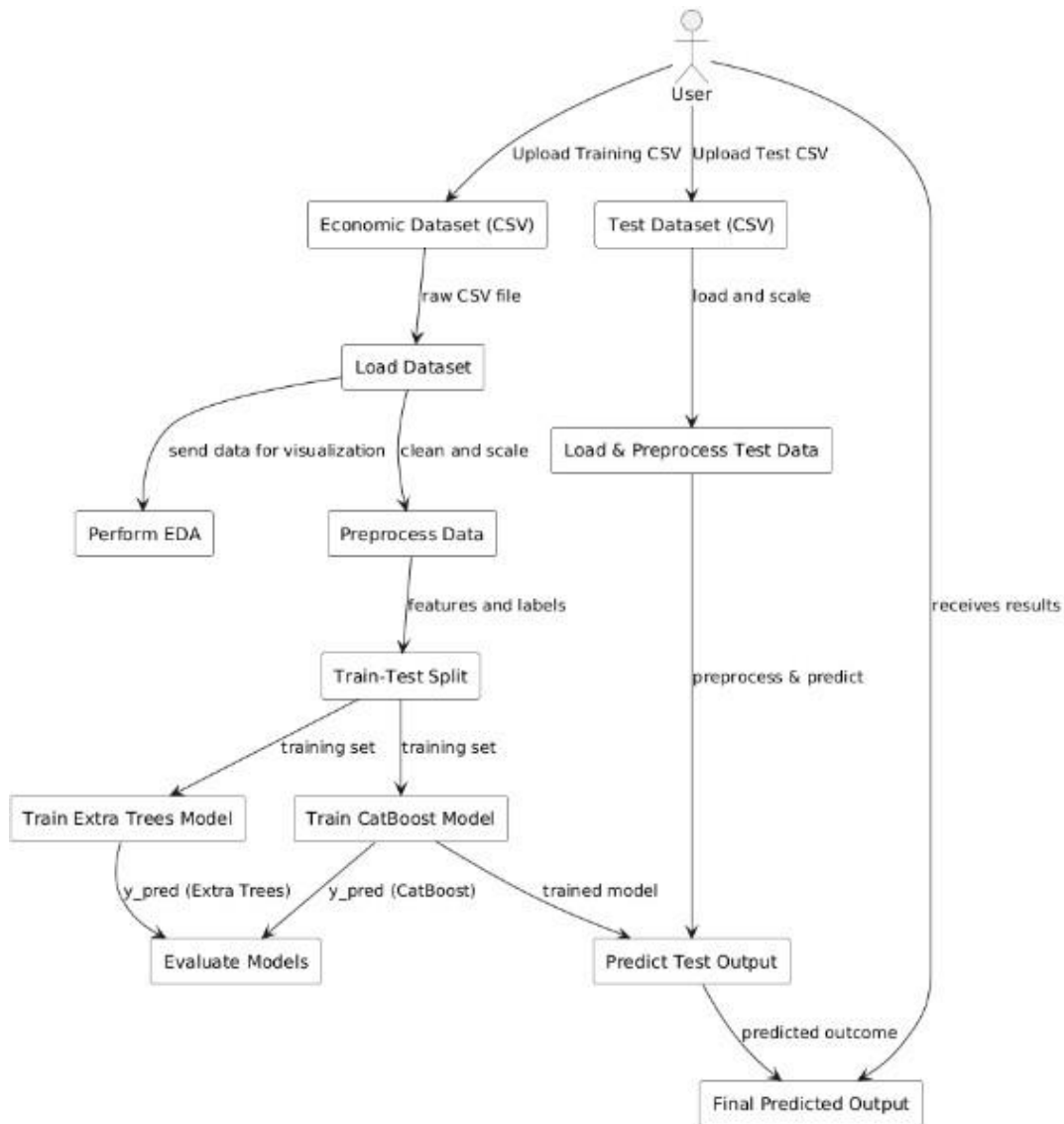


Fig. 5.2: Dataflow Diagram.

5.3 Sequence Diagram

A UML sequence diagram is an interaction diagram that illustrates how processes or objects interact with each other in a specific order. It visually maps this cooperation using parallel vertical lines, called "lifelines," to represent the different processes or objects existing over time, and horizontal arrows to depict the messages exchanged between them chronologically. This structure provides a clear, graphical method for specifying simple runtime scenarios and understanding the flow of control in a system.

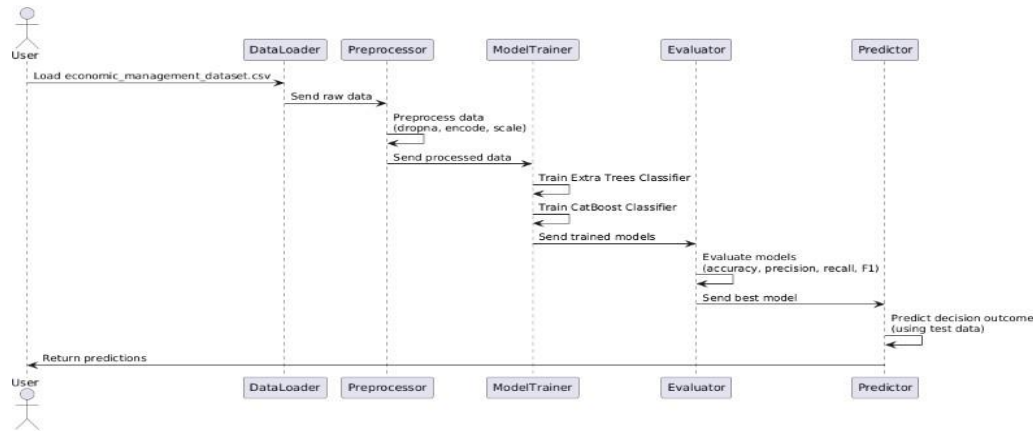


Fig. 5.3: Sequence Diagram

5.4 Activity diagram

Activity diagram is another important diagram in UML to describe the dynamic aspects of the system.

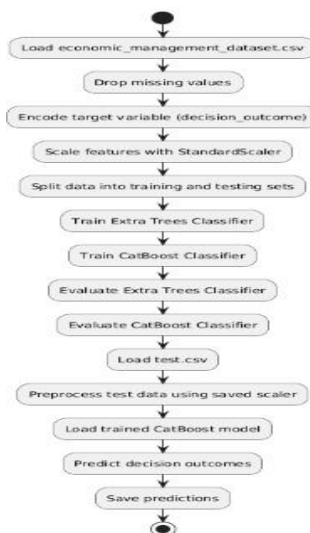


Fig. 5.4: Activity Diagram.

5.5 Deployment diagram: The deployment diagram visualizes the physical hardware on which the software will be deployed.

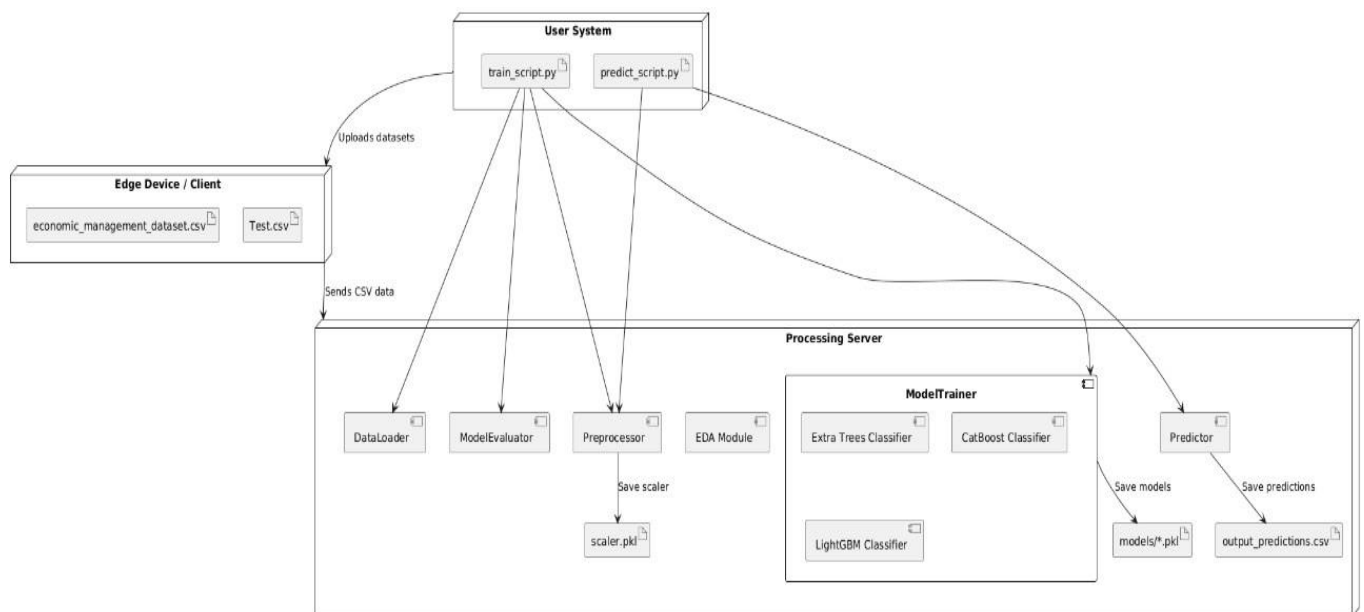


Fig. 5.5: Deployment Diagram.

5.6 Use case diagram: The purpose of use case diagram is to capture the dynamic aspect of a system.

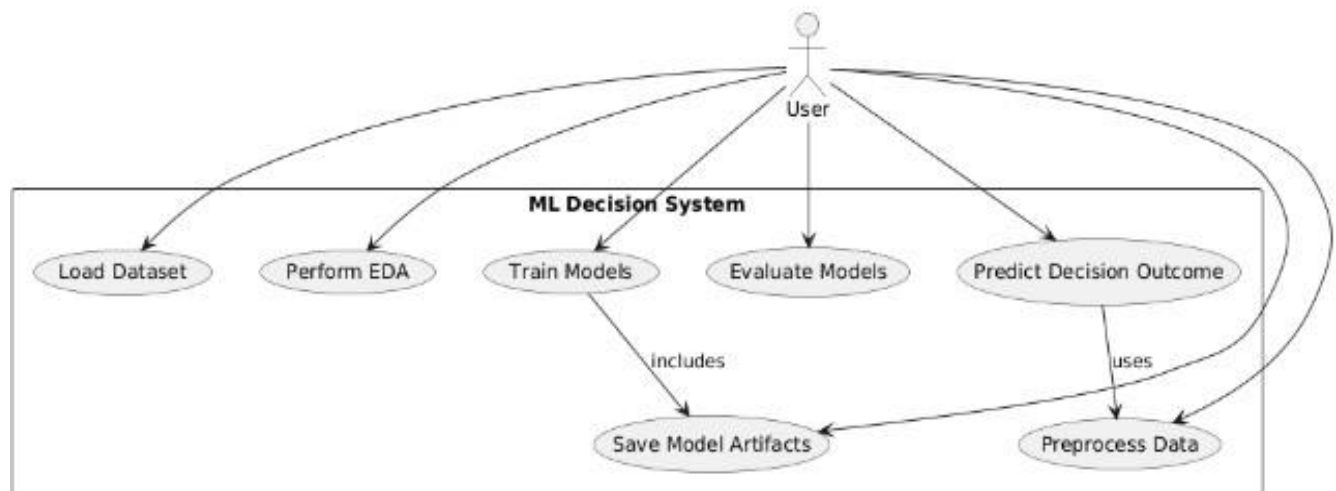


Fig. 5.6: Use Case Diagram.

5.7 Component Diagram

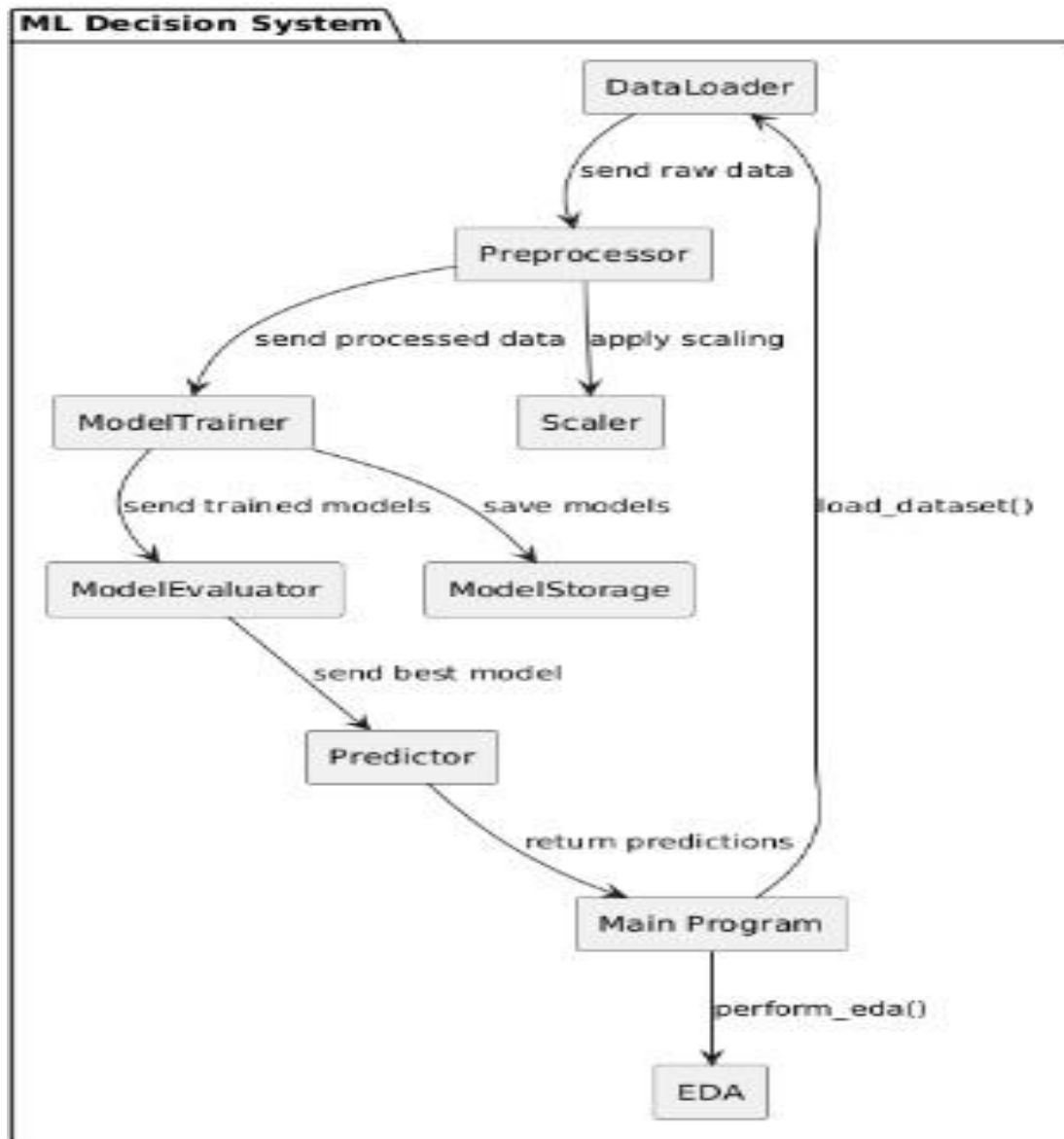


Fig. 5.7: Component Diagram

CHAPTER 6

SOFTWARE REQUIREMENTS

6.1 Software Requirements

Python is a high-level, interpreted programming language known for its simplicity and readability, which makes it a popular choice for beginners as well as experienced developers. Key features of Python include its dynamic typing, automatic memory management, and a rich standard library that supports a wide range of applications from web development to data science and machine learning. Its object-oriented approach and support for multiple programming paradigms allow developers to write clear, maintainable code. Python's extensive ecosystem of third-party packages further enhances its capabilities, enabling rapid development and prototyping across diverse fields.

Installation

First, download the appropriate installer from the official Python website (<https://www.python.org/downloads/release/python-376/>). For Windows users, run the executable installer and ensure to check the "Add Python to PATH" option during installation; for macOS and Linux, follow the respective package installation commands or use a package manager like Homebrew or apt-get. After installation, verify the setup by running `python --version` or `python3 --version` in your terminal or command prompt, which should display "Python 3.7.6." This version-specific installation supports all major functionalities and libraries compatible with Python 3.7.6, making it an excellent foundation for developing robust applications in areas such as data analysis, machine learning, and GUI development.

6.1.1 Python Packages

The project requires a robust set of software libraries and tools that work together to build an integrated system for plant disease classification. Below is an explanation of the key software requirements and the packages used:

- **Python:** The project is implemented in Python, which is chosen for its extensive ecosystem of libraries and its strong support for data analysis, machine learning, and GUI development.
- **Tkinter:** Used to build the graphical user interface (GUI) of the application. It handles tasks user authentication, data upload, and displaying results, making accessible to both admins and end-users.

- **PIL (Pillow):** Utilized for image processing, particularly for handling background images and other graphical elements within the GUI, thereby enhancing the visual appeal of the application.
- **Matplotlib & Seaborn:** These libraries are employed for data visualization. Matplotlib is used for creating standard plots, while Seaborn adds an extra layer of sophistication for statistical visualizations such as bar plots, violin plots, histograms, scatter plots, strip plots, and correlation heat maps.
- **Pandas & NumPy:** Essential for data manipulation and analysis. Pandas is used to load, preprocess, and analyze the CSV dataset, while NumPy supports numerical operations and data handling, which are crucial for processing large volumes of IoT data.
- **Scikit-learn (sklearn):** Provides the machine learning framework used in the project. It includes tools for model training, evaluation, train-test splitting, and data preprocessing (like label encoding). Models such as Gaussian Naive Bayes, SVM, KNN, and Decision Tree Classifier are implemented using scikit-learn.
- **Imbalanced-learn (imblearn):** Specifically used for implementing the SMOTE (Synthetic Minority Oversampling Technique) algorithm, which helps in addressing class imbalance in the dataset by generating synthetic samples for under-represented classes.
- **Joblib:** Utilized for saving and loading trained machine learning models. This ensures that once a model is trained, it can be stored and reused without retraining, thereby improving efficiency.
- **PyMySQL:** This package provides a means to connect to a MySQL database for handling user authentication. It facilitates operations such as user signup, login, and data storage, ensuring secure and persistent management of user credentials.

Each of these packages plays a crucial role in ensuring that the system is robust, scalable, and efficient—from data ingestion and preprocessing to model training, visualization, and deployment. The combination of these tools enables the creation of an integrated, user-friendly application for real-time plant disease classification and management.

6.2 Hardware Requirements

Python 3.7.6 can run efficiently on most modern systems with minimal hardware requirements. However, meeting the recommended specifications ensures better performance, especially for developers handling large-scale applications or computationally intensive tasks. By ensuring compatibility with hardware and operating system, can leverage the full potential of Python 3.7.6.

Processor (CPU) Requirements: Python 3.7.6 is a lightweight programming language that can run on various processors, making it highly versatile. However, for optimal performance, the following processor specifications are recommended:

- **Minimum Requirement:** 1 GHz single-core processor.
- **Recommended:** Dual-core or quad-core processors with a clock speed of 2 GHz or higher. Using a multi-core processor allows Python applications, particularly those involving multithreading or multiprocessing, to execute more efficiently.

Memory (RAM) Requirements: Python 3.7.6 does not demand excessive memory but requires adequate RAM for smooth performance, particularly for running resource-intensive applications such as data processing, machine learning, or web development.

- **Minimum Requirement:** 512 MB of RAM.
- **Recommended:** 4 GB or higher for general usage. For data-intensive operations, 8 GB or more is advisable.

Insufficient RAM can cause delays or crashes when handling large datasets or executing computationally heavy programs.

Storage Requirements: Python 3.7.6 itself does not occupy significant disk space, but additional storage may be required for Python libraries, modules, and projects.

- **Minimum Requirement:** 200 MB of free disk space for installation.
- **Recommended:** At least 1 GB of free disk space to accommodate libraries and dependencies.

Developers using Python for large-scale projects or data science should allocate more storage to manage virtual environments, datasets, and frameworks like TensorFlow or PyTorch.

Compatibility with Operating Systems: Python 3.7.6 is compatible with most operating systems but requires hardware that supports the respective OS. Below are general requirements for supported operating systems:

- **Windows:** 32-bit and 64-bit systems, Windows 7 or later.
- **macOS:** macOS 10.9 or later.
- **Linux:** Supports a wide range of distributions, including Ubuntu, CentOS, and Fedora.

The hardware specifications for the OS directly impact Python's performance, particularly for modern software development.

CHAPTER 7

FUNCTIONAL REQUIREMENTS

1. Dataset Loading

- **FR1.1:** The system shall allow users to specify the path to a CSV file containing economic decision data.
- **FR1.2:** The system shall load the CSV file into a Data Frame using pandas.
- **FR1.3:** The system shall handle and report errors if the file is not found or improperly formatted.

2. Data Preprocessing

- **FR2.1:** The system shall drop any rows in the dataset containing missing values (NaN).
- **FR2.2:** The system shall map the target variable decision outcome:
 - 'Effective' → 1
 - 'Not Effective' → 0
- **FR2.3:** The system shall separate the dataset into feature variables (X) and target variable (y) during training.
- **FR2.4:** The system shall apply standard scaling to the feature variables using Standard Scaler.
- **FR2.5:** The system shall save the fitted scaler to the models/ directory for consistent preprocessing of test data.

3. Exploratory Data Analysis (EDA)

- **FR3.1:** The system shall generate the following plots:
 - Correlation heatmap of all numeric features.
 - Bar plot for distribution of decision outcome.
 - Box plot for transaction volume by decision outcome.
 - Scatter plot of financial metric vs system throughput.
 - Histogram for resource utilization.
- **FR3.2:** The system shall save all plots into a directory (eda plots/).

4. Dataset Splitting

- **FR4.1:** The system shall split the preprocessed dataset into training and testing sets.
- **FR4.2:** The system shall allow configuring the test size (default is 20%).
- **FR4.3:** The system shall set a random seed (random state=42) to ensure reproducibility.

5. Model Training

- **FR5.1:** The system shall train the following classifiers using the training set:
 - Extra Trees Classifier
 - Cat Boost Classifier
 - (LightGBM is defined but not used in this run)
- **FR5.2:** The system shall store each trained model in the models/ directory using joblib.
- **FR5.3:** If a model file already exists, the system shall load it instead of retraining.

6. Model Evaluation

- **FR6.1:** The system shall use the testing set to evaluate each trained model.
- **FR6.2:** The system shall compute the following metrics:
 - Accuracy
 - Precision
 - Recall
 - F1-score
- **FR6.3:** The system shall generate and save the confusion matrix plot for each model in the plots/ directory.
- **FR6.4:** The system shall print a classification report including performance per class (Effective, Not Effective).

7. Test Data Prediction

- **FR7.1:** The system shall accept an unseen/test dataset (CSV) as input.
- **FR7.2:** The system shall preprocess the test dataset using the same scaler saved during training.

- **FR7.3:** The system shall allow selecting the prediction model by name (e.g., 'Extra Trees', 'Cat Boost').
- **FR7.4:** The system shall predict decision outcome values for the test dataset.
- **FR7.5:** The system shall map prediction values:
 - $1 \rightarrow \text{'Effective'}$
 - $0 \rightarrow \text{'Not Effective'}$
- **FR7.6:** The system shall return the test dataset with a new column predicted decision outcome.

8. File Management & Directories

- **FR8.1:** The system shall create the following directories if they do not exist:
 - models/ (for saving models and scaler)
 - plots/ (for confusion matrices)
 - eda plots/ (for EDA visualizations)

CHAPTER 8

SOURCE CODE

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import os


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report,
confusion_matrix

from sklearn.ensemble import ExtraTreesClassifier

from catboost import CatBoostClassifier

from lightgbm import LGBMClassifier

import joblib


# Create model directory if it doesn't exist

MODEL_DIR = 'models'

if not os.path.exists(MODEL_DIR):

    os.makedirs(MODEL_DIR)

def load_dataset(file_path):
```



```

"""Load the dataset from CSV file."""

try:

    df = pd.read_csv(file_path)

    return df

except Exception as e:

    print(f"Error loading dataset: {e}")

    return None

def preprocess_data(df, is_train=True):

    """Preprocess the dataset: handle missing values, encode target, and scale features."""

    # Create a copy to avoid modifying the original dataframe

    df = df.copy()

    # Handle missing values

    df = df.dropna()

    # If training data, encode target variable

    if is_train:

        df['decision_outcome'] = df['decision_outcome'].map({'Effective': 1, 'Not Effective': 0})

    # Separate features and target

    if is_train:

        X = df.drop('decision_outcome', axis=1)

        y = df['decision_outcome']

    else:

```

```

X = df

y = None

# Scale numerical features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

# Save scaler for test data

if is_train:

    joblib.dump(scaler, os.path.join(MODEL_DIR, 'scaler.pkl'))

return X_scaled, y, scaler if is_train else X_scaled

def perform_eda(df, save_path='eda_plots'):

    """Perform Exploratory Data Analysis and generate 5 plots."""

    if not os.path.exists(save_path):

        os.makedirs(save_path)

    # 1. Correlation Heatmap

    plt.figure(figsize=(10, 8))

    sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')

    plt.title('Correlation Heatmap of Features')

    plt.savefig(os.path.join(save_path, 'correlation_heatmap.png'))

    plt.show()

```

2. Distribution of Decision Outcome

```
plt.figure(figsize=(8, 6))

sns.countplot(x='decision_outcome', data=df)

plt.title('Distribution of Decision Outcome')

plt.savefig(os.path.join(save_path, 'decision_outcome_distribution.png'))

plt.show()
```

3. Box Plot of Transaction Volume by Decision Outcome

```
plt.figure(figsize=(8, 6))

sns.boxplot(x='decision_outcome', y='transaction_volume', data=df)

plt.title('Transaction Volume by Decision Outcome')

plt.savefig(os.path.join(save_path, 'transaction_volume_boxplot.png'))

plt.show()
```

4. Scatter Plot of Financial Metric vs System Throughput

```
plt.figure(figsize=(8, 6))

sns.scatterplot(x='financial_metric', y='system_throughput', hue='decision_outcome', data=df)

plt.title('Financial Metric vs System Throughput')

plt.savefig(os.path.join(save_path, 'financial_metric_vs_throughput.png'))

plt.show()
```

```

# 5. Histogram of Resource Utilization

plt.figure(figsize=(8, 6))

sns.histplot(df['resource_utilization'], bins=30, kde=True)

plt.title('Distribution of Resource Utilization')

plt.savefig(os.path.join(save_path, 'resource_utilization_histogram.png'))

plt.show()

def train_test_split_data(X, y, test_size=0.2, random_state=42):

    """Split the dataset into training and testing sets."""

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size, random_state=random_state)

    return X_train, X_test, y_train, y_test

def evaluate_performance(y_true, y_pred, model_name, plot_cm=True, save_path='plots'):

    """Evaluate model performance and plot confusion matrix."""

    if not os.path.exists(save_path):

        os.makedirs(save_path)

    # Calculate metrics

    accuracy = accuracy_score(y_true, y_pred)

    precision = precision_score(y_true, y_pred)

    recall = recall_score(y_true, y_pred)

    f1 = f1_score(y_true, y_pred)

    # Print metrics

    print(f"\nPerformance Metrics for {model_name}:")

```

```

print(f'Accuracy (A): {accuracy:.4f}')

print(f'Precision (P): {precision:.4f}')

print(f'Recall (R): {recall:.4f}')

print(f'F1 Score: {f1:.4f}')

# Print classification report

print(f'\nClassification Report for {model_name}:')

print(classification_report(y_true, y_pred, target_names=['Not Effective', 'Effective']))

# Plot confusion matrix

if plot_cm:

    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(8, 6))

    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Not Effective', 'Effective'],
yticklabels=['Not Effective', 'Effective'])

    plt.title(f'Confusion Matrix for {model_name}')

    plt.ylabel('True Label')

    plt.xlabel('Predicted Label')

    plt.savefig(os.path.join(save_path, f'{model_name}_confusion_matrix.png'))

    plt.show()

    return {'Accuracy': accuracy, 'Precision': precision, 'Recall': recall, 'F1 Score': f1}

def train_extra_trees(X_train, y_train, model_path=os.path.join(MODEL_DIR, 'extra_trees.pkl')):

    """Train and save Extra Trees Classifier."""

```

```

if os.path.exists(model_path):

    model = joblib.load(model_path)

    print("Loaded existing Extra Trees model")

else:

    model = ExtraTreesClassifier(n_estimators=100, random_state=42)

    model.fit(X_train, y_train)

    joblib.dump(model, model_path)

    print("Trained and saved Extra Trees model")

return model

def train_catboost(X_train, y_train, model_path=os.path.join(MODEL_DIR, 'catboost.pkl')):

    """Train and save CatBoost Classifier."""

    if os.path.exists(model_path):

        model = joblib.load(model_path)

        print("Loaded existing CatBoost model")

    else:

        model = CatBoostClassifier(iterations=100, depth=6, learning_rate=0.1, random_seed=42, verbose=0)

        model.fit(X_train, y_train)

        joblib.dump(model, model_path)

        print("Trained and saved CatBoost model")

    return model

def train_lightgbm(X_train, y_train, model_path=os.path.join(MODEL_DIR, 'lightgbm.pkl')):

```

```

"""Train and save LightGBM Classifier."""

if os.path.exists(model_path):

    model = joblib.load(model_path)

    print("Loaded existing LightGBM model")

else:

    model = LGBMClassifier(n_estimators=100, learning_rate=0.1, random_state=42)

    model.fit(X_train, y_train)

    joblib.dump(model, model_path)

    print("Trained and saved LightGBM model")

return model

def predict_test_data(test_file_path, model_name='extra_trees'):

    """Preprocess test data and make predictions using the specified model."""

    # Load test data

    test_df = load_dataset(test_file_path)

    if test_df is None:

        return None

    # Preprocess test data (unpack features)

    X_test_scaled = preprocess_data(test_df, is_train=False)[0]

    # Load model

    model_path = os.path.join(MODEL_DIR, f'{model_name.lower()}.pkl')

```

```

if not os.path.exists(model_path):

    print(f'Model {model_name} not found!')

    return None

model = joblib.load(model_path)

# Make predictions

predictions = model.predict(X_test_scaled)

# Map predictions back to original categories

predictions_mapped = np.where(predictions == 1, 'Effective', 'Not Effective')

# Create output dataframe

output_df = test_df.copy()

output_df['predicted_decision_outcome'] = predictions_mapped

return output_df

train_file_path = 'economic_management_dataset.csv'

df = load_dataset(train_file_path)

df

perform_eda(df)

X, y, _ = preprocess_data(df)

X_train, X_test, y_train, y_test = train_test_split_data(X, y)

models = {

    'Extra Trees': train_extra_trees(X_train, y_train),

```



```
    'CatBoost': train_catboost(X_train, y_train) }

results = {}

for name, model in models.items():

    y_pred = model.predict(X_test)

    results[name] = evaluate_performance(y_test, y_pred, name)

test_file_path = 'Test.csv' # Replace with actual test file path

output_df = predict_test_data(test_file_path, model_name='CatBoost')

output_df
```

CHAPTER 9

RESULTS AND DISCUSSION

1. Importing Required Libraries

- Prepare the coding environment with all necessary tools.
- Imports data manipulation libraries like `pandas` and `numpy`.
- Visualization tools such as `matplotlib` and `seaborn` are included to explore and visualize the data.
- Scikit-learn modules are used for data preprocessing (like encoding and scaling), model training, and performance evaluation.
- `joblib` is used for saving and loading trained models.
- Warnings are suppressed to keep the console output tidy and focused.

2. Data Loading and Basic Exploration

- Understand the structure and quality of the dataset.
- The dataset is loaded from a CSV file into a `DataFrame`.
- Initial inspection is performed using functions to:
- Display data types and structure (`info()`),
- Check for missing values (`isnull().sum()`),
- Identify duplicate records (`duplicated().sum()`).
- A correlation heatmap is plotted to visually understand how features relate to each other.

3. Data Preprocessing

- Prepare the dataset for modeling by encoding and splitting.
- Categorical columns such as `sex`, `smoker`, and `region` are transformed into numeric form using label encoding.
- The dataset is divided into:
- **Features (X)**: All independent variables.
- **Target (y)**: The dependent variable `charges`.
- The data is split into training and testing sets in an 80:20 ratio to allow for model validation.

4. Performance Metrics Function

- Standardize how model performance is measured.
- A custom function `performance_metrics()` is defined to evaluate predictions using:
 - Mean Absolute Error (MAE),
 - Mean Squared Error (MSE),
 - Root Mean Squared Error (RMSE),
 - R^2 Score.
- This function also generates a scatter plot to visually compare the actual vs. predicted values.

5. K-Nearest Neighbors (KNN) Regressor

- Train or reuse a model to predict insurance charges.
- The script first checks if a previously saved KNN model exists.
 - If it exists: Loads the model and makes predictions.
 - If it doesn't: Initializes, trains a new model on the training data, saves it, and makes predictions.
- Predictions are evaluated using the `performance_metrics()` function.

6. Gradient Boosting Regressor

- Build an alternative, more complex model for prediction.
- Similarly, the script checks for a saved Gradient Boosting model.
 - If found: Loads and uses it for prediction.
 - If not: Trains a new model, saves it, and performs prediction.
- Again, evaluation is performed using the same performance metrics.

7. Prediction on New Test Data

- Apply the trained model to unseen data for real-world use.
- A new test dataset (`testdata.csv`) is loaded.
- Any unnecessary or irrelevant columns are dropped.
- The trained Gradient Boosting model is used to predict `charges`.

- The results are added as a new column to the dataset, making it ready for export, reporting, or further analysis.

9.1 Dataset Description

1. Transaction volume

- Represents the total number or volume of transactions handled by the system within a given timeframe.
- High values might indicate a busy or active environment, and the model could relate this to system load or decision efficiency.

2. Market behavior index

- A calculated indicator reflecting market trends or behavioral patterns.
- Could be derived from market volatility, sentiment, or consumer activity. Useful for correlating external economic behavior with decision performance.

3. Financial metric

- Denotes financial performance indicators like ROI, profit margin, or cost efficiency.
- Helps the model evaluate how economic factors influence or correlate with decision effectiveness.

4. Edge processing latency

- Measures the time taken by edge devices to process data and return results.
- Lower latency generally contributes to faster and more responsive systems. Important in real-time decision-making systems.

5. Resource utilization

- Indicates the percentage of system resources (CPU, memory, bandwidth) being used.
- High utilization might strain the system, possibly degrading decision accuracy or throughput.

6. **Workload distribution efficiency**

- Reflects how evenly and effectively computational tasks are distributed across edge and cloud nodes.
- Efficient distribution helps prevent bottlenecks and improves system resilience and performance.

7. **Decision accuracy**

- Represents how correct or effective previous decisions were, possibly based on actual outcomes versus predicted.
- Acts as a historical feedback loop to assess decision-making quality.

8. **System throughput**

- Denotes the rate at which the system successfully processes requests or transactions per unit time.
- A critical measure for evaluating system performance under real-time load.

9. **Decision outcome**

- The target variable in your model; categorizes whether a decision was “**Effective**” or “**Not Effective**”.
- This binary label helps train classification models to predict future decision effectiveness based on all other features.

9.2 Result Analysis

Fig. 9.2.(a) presents a snapshot of the dataset utilized for developing a real-time economic decision-making model powered by edge computing. The dataset captures a range of system and market attributes including transaction volume, market behavior index, financial metrics, edge processing latency, resource utilization, and workload distribution efficiency. Additionally, the dataset records decision accuracy, which serves as a performance indicator of prior decisions. With 500 records and 9 key features, this structured dataset enables the application of machine learning algorithms to discover complex relationships and patterns. The

predictive modeling aims to estimate or classify decision outcomes in economic environments, especially under edge computing scenarios, by leveraging operational efficiency, resource usage, and market behavior patterns.

	transaction_volume	market_behavior_index	financial_metric	edge_processing_latency	resource_utilization	workload_distribution_efficiency	decision_accura
0	1717	-0.702675	74.296381	472.107160	0.947553	78.856129	84.2176
1	3782	-0.170752	42.606784	127.939429	0.701715	67.856058	91.4919
2	5946	-0.829301	34.658906	69.535674	0.710803	89.377454	78.1282
3	6486	0.993749	37.166772	106.760540	0.425728	62.550069	76.0676
4	9335	0.004390	98.766191	444.593201	0.634295	78.203701	79.4148
...	...	...	...	...	...	...	...
495	3713	-0.938695	22.539100	215.284417	0.434552	82.706637	90.5159
496	6588	-0.484634	37.279486	378.077243	0.453212	94.456396	85.9471
497	1966	-0.074754	43.264481	44.793756	0.778838	78.940338	70.9797
498	5944	0.736545	43.996558	49.288207	0.926584	81.619566	84.2678
499	4432	0.454338	61.332664	183.837933	0.955836	57.823836	82.2324

500 rows x 9 columns

Fig. 9.2.(a): Sample Dataset

Fig. 9.2.(b) The correlation heatmap visualizes relationships among key features affecting real-time decision-making systems. Most variables exhibit weak correlations, indicating minimal redundancy in the dataset. Notable patterns include slight positive links between transaction volume and edge latency, and between workload efficiency and decision accuracy. This independence among features supports their collective use in predictive modeling for accurate system performance evaluation.

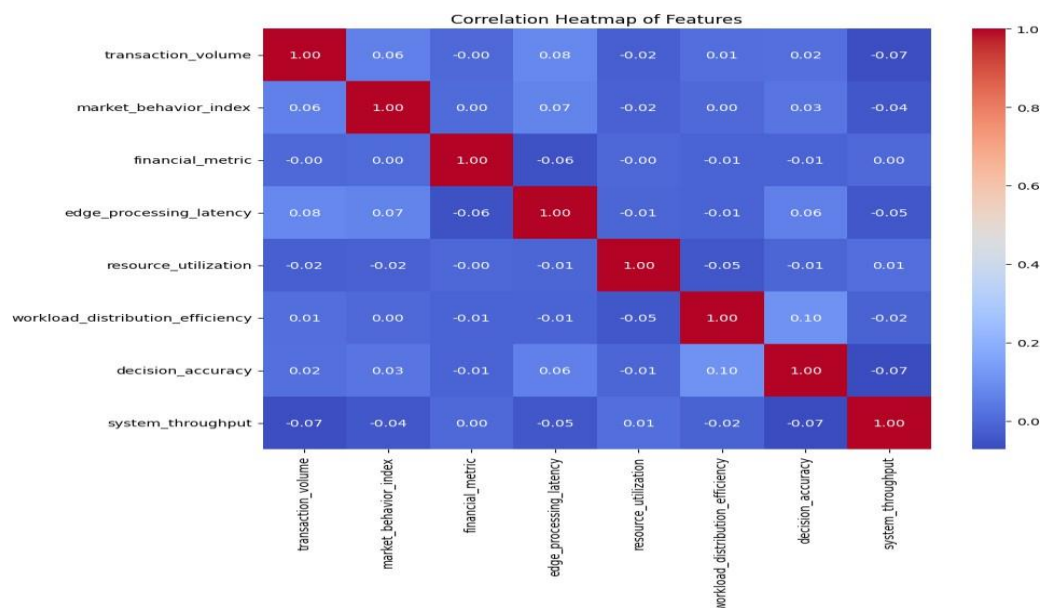


Fig. 9.2 (b): Correlation Matrix of System and Market Parameters

Fig. 9.2(c)The bar chart illustrates the distribution of decision outcomes, showing a slightly higher count for "Not Effective" decisions compared to "Effective" ones. This near-balanced distribution suggests that the dataset is well-suited for classification tasks. It highlights the variability in decision performance, which can be useful for training models to improve the effectiveness of automated or assisted decision-making systems.

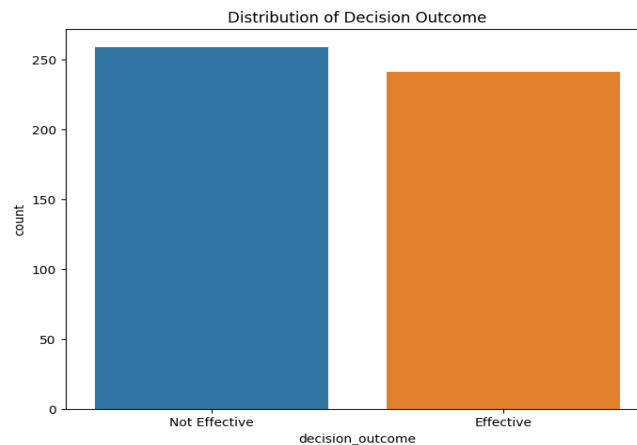


Fig. 9.2. (c): Distribution of Decision Outcome Categories.

Fig. 9.2(d) The box plot compares transaction volumes across decision outcomes—"Effective" and "Not Effective." Both categories exhibit a wide range with similar medians, though the "Effective" outcome shows slightly higher upper quartile values. This visualization reveals that higher transaction volumes do not necessarily ensure more effective decisions, highlighting the potential need for deeper analysis of influencing factors beyond transaction count.

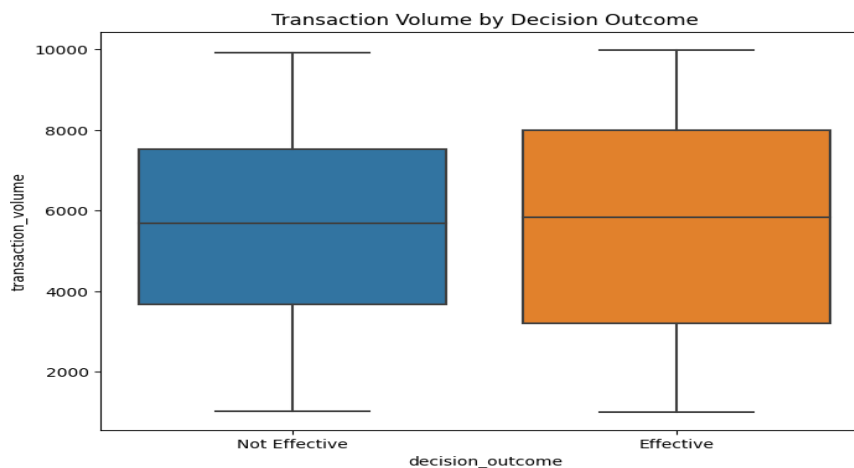


Fig. 9.2 (d): Transaction Volume Distribution by Decision Outcome

Fig. 9.2(e) The scatter plot visualizes the relationship between financial metrics and system throughput, categorized by decision outcome. The data points are widely dispersed, indicating no strong linear correlation between the two variables. Both “Effective” and “Not Effective” decisions appear throughout the entire range, suggesting that decision effectiveness is influenced by more complex or additional factors beyond just financial performance and throughput.

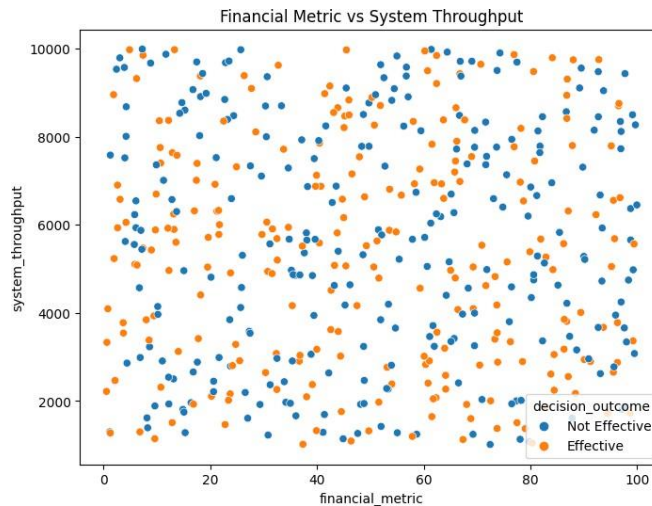


Fig. 9.2 (e): Financial Metric vs System Throughput by Decision Outcome.

Fig. 9.2(f) The histogram displays the distribution of resource utilization values across the dataset. The data appears relatively uniform, with values spanning the entire range from 0 to 1. Minor peaks and dips suggest slight variability in usage intensity, but overall, resource utilization is well-distributed. The KDE curve highlights modest fluctuations without indicating any dominant concentration or skewness in utilization patterns.

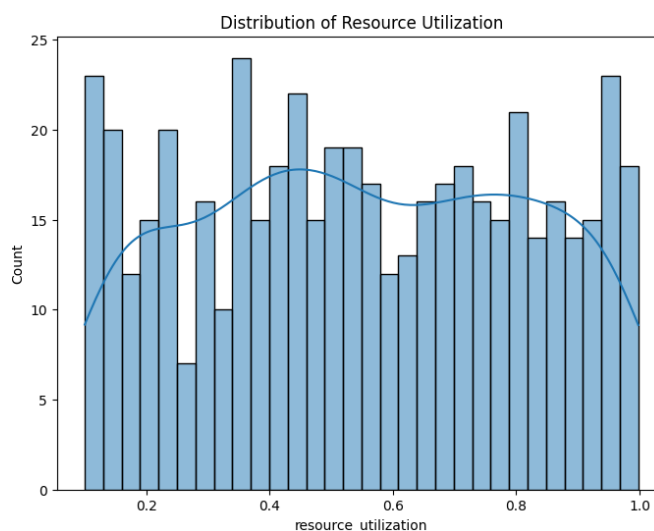


Fig. 9.2 (f): Distribution of Resource Utilization

Fig. 9.2(g) The figure presents a sample of predicted decision outcomes generated by the trained model on test data. The outcomes are classified into two categories: “Effective” and “Not Effective.” The model predicts a mix of both classes, indicating its ability to distinguish decision effectiveness. This distribution helps evaluate the model's performance in real-world economic decision-making scenarios.

predicted_decision_outcome
Not Effective
Not Effective
Not Effective
Not Effective
Not Effective
Not Effective
Not Effective
Effective
Effective
Effective
Effective
Effective
Effective

Fig. 9.2 (g): Predictions on test data.

9.3 Comparative Analysis

Table. 9.1: Comparison of existing Extra Trees and Cat Boost Classifier.

Metric	Extra Trees Classifier	CatBoost Classifier
Accuracy	0.9700	1.0000
Precision	1.0000	1.0000
Recall	0.9412	1.0000
F1 Score	0.9697	1.0000

Table. 9.1 The comparative evaluation of the Extra Trees and CatBoost classifiers shows that both models performed exceptionally well on the test data, but CatBoost slightly outperformed Extra Trees across all metrics. The Extra Trees model achieved an accuracy of 97%, with a perfect precision of 1.00, indicating it made no false positive predictions. However, its recall of 0.9412 implies it missed a small fraction of actual "Effective" cases, which is reflected in its F1-score of 0.9697—a balance between precision and recall. In contrast, the CatBoost classifier achieved a perfect score across all performance metrics, including 100% accuracy, precision, recall, and F1-score. This suggests that CatBoost was able to correctly classify all instances of both “Effective” and “Not Effective” outcomes, making it the more reliable model for predicting decision effectiveness in this context.

CHAPTER 10

CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

In conclusion, both the Extra Trees and CatBoost classifiers demonstrated strong predictive performance in evaluating decision outcomes. The Extra Trees model achieved a high accuracy of 97%, with excellent precision and F1-score values, making it a robust model with minimal misclassifications. However, it slightly underperformed in recall, indicating a few missed instances of the "Effective" class. On the other hand, the CatBoost classifier delivered perfect classification results across all key metrics—accuracy, precision, recall, and F1-score—each scoring 1.00. This flawless performance highlights CatBoost's superior capability in handling the given dataset, especially in achieving a perfect balance between identifying positive outcomes and avoiding false predictions. Therefore, while Extra Trees is commendable, CatBoost stands out as the most reliable model for predicting the effectiveness of decisions, ensuring both precision and comprehensive coverage. Its exceptional accuracy makes it highly suitable for real-time economic decision-making applications involving edge computing systems.

10.2 Future scope

The future scope of empowering real-time economic decision-making through edge computing for minimal latency is highly promising. As industries increasingly rely on instant data processing, edge computing will become essential for enabling faster, more efficient, and localized economic decisions. The integration of edge computing with AI and machine learning will allow predictive and autonomous decision-making at the data source, reducing reliance on central cloud systems. Advancements in edge hardware and 5G connectivity will further enhance responsiveness and scalability. Additionally, improved data privacy, reduced operational costs, and support for dynamic pricing, fraud detection, and demand forecasting will drive adoption across sectors such as finance, retail, and manufacturing. Edge computing also aligns with sustainability goals by minimizing data transfer and energy use. Overall, the technology holds the potential to transform economic systems by enabling secure, real-time, and context-aware decision-making.

REFERENCES

- [1].Premasankar, G.; Di Francesco, M.; Taleb, T. Edge computing for the Internet of Things: A case study. *IEEE Internet Things J.* **2018**, *5*, 1275–1284.
- [2].Björkdahl, J. Strategies for digitalization in manufacturing firms. *Calif. Manag. Rev.* **2020**, *62*, 17–36.
- [3].Stojmenovic, I. Fog computing: A cloud to the ground support for smart things and machine-to-machine networks. In Proceedings of the 2014 Australasian Telecommunication Networks and Applications Conference (ATNAC), Melbourne, Australia, 26–28 November 2014; pp. 117–122.
- [4].Oh, Y.; Park, H.; Yoo, A.; Kim, N.; Kim, Y.; Kim, D.; Choi, J.; Yoon, S.; Yang, H. A Product Quality Prediction Model Using Real-Time Process Monitoring in Manufacturing Supply Chain. *J. Korean Inst. Ind. Eng.* **2013**, *39*, 271–277.
- [5].Zhu, Z. Research on the Application of Real-Time Monitoring System for Manufacturing Quality of Industrial Production Based on Industrial 4.0. In *Lecture Notes in Electrical Engineering*; Springer: Singapore, 2017.
- [6].Jiao, L.; Friedman, R.; Fu, X.; Secci, S.; Smoreda, Z.; Tschofenig, H. Cloud-based computation offloading for mobile devices: State of the art, challenges and opportunities. In Proceedings of the 2013 Future Network & Mobile Summit, Lisboa, Portugal, 3–5 July 2013; pp. 1–11.
- [7].Barboni, L. Machine Learning and Edge Computing for Industry 4.0 Applications: Concepts and Extensive Review. In *Innovation and Competitiveness in Industry 4.0 Based on Intelligent Systems*; Springer: Cham, Switzerland, 2023; pp. 3–19.
- [8].Willner, A.; Gowtham, V. Toward a reference architecture model for industrial edge computing. *IEEE Commun. Stand. Mag.* **2020**, *4*, 42–48.
- [9].Veith, A.; Assunção, M.; Lefèvre, L. *Latency-Aware Placement of Data Stream Analytics on Edge Computing*; Springer: Cham, Switzerland, 2018; pp. 215–229.
- [10]. Aral, A.; Brandić, I.; Uriarte, R.B.; Nicola, R.D.; Scoca, V. Addressing Application Latency Requirements through Edge Scheduling. *J. Grid Comput.* **2019**, *17*, 677–698.
- [11]. Zhang, Y.; Pang, C.; Yang, G. A Real-time Computation Task Reconfiguration Mechanism for Industrial Edge Computing. In Proceedings of the IECON 2020 the 46th Annual Conference of the IEEE Industrial Electronics Society, Singapore, 18–21 October 2020; pp. 3799–3804.

- [12]. Bacchiani, L.; Palma, G.; Sciullo, L.; Bravetti, M.; Felice, M.D.; Gabbrielli, M.; Zavattaro, G.; Penna, R.D. Low-Latency Anomaly Detection on the Edge-Cloud Continuum for Industry 4.0 Applications: The SEAWALL Case Study. *IEEE Internet Things Mag.* **2022**, *5*, 32–37.
- [13]. Abouaomar, A.; Cherkaoui, S.; Mlika, Z.; Kobbane, A. Resource Provisioning in Edge Computing for Latency-Sensitive Applications. *IEEE Internet Things J.* **2021**, *8*, 11088–11099.
- [14]. Jo, J.; Jeong, S.; Kang, P. Benchmarking gpu-accelerated edge devices. In Proceedings of the 2020 IEEE International Conference on Big Data and Smart Computing (BigComp), Busan, Republic of Korea, 19–22 February 2020; pp. 117–120.
- [15]. Hussain, R.F.; Salehi, M.A. Resource allocation of industry 4.0 micro-service applications across serverless fog federation. *Future Gener. Comput. Syst.* **2024**, *154*, 479–490.
- [16]. Shi, Weisong, et al. Edge Computing: Vision and Challenges. *IEEE IoT Journal*, 2016.
- [17]. Satyanarayanan, Mahadev. The Emergence of Edge Computing. *IEEE Computer*, 2017.
- [18]. Varghese, Blessen, et al. Challenges and Opportunities in Edge Computing. *IEEE Cloud Computing*, 2016.
- [19]. Hu, Yong, et al. Mobile Edge Computing—A Key Technology towards 5G. *IEEE Network*, 2015.
- [20]. Dinh, Hoang T., et al. A Survey of Mobile Cloud Computing: Architecture, Applications, and Approaches. *Wireless Communications and Mobile Computing*, 2013.
- [21]. Yi, Shanhe, et al. Fog Computing: Platform and Applications. 2015.
- [22]. Mahmoud, Magdy M., et al. Security for Edge Computing. *IEEE Communications Magazine*, 2018.
- [23]. Deng, Shancang, et al. Edge Computing: A Survey on Architecture and Security. *IEEE Communications Surveys & Tutorials*, 2020.
- [24]. Zhang, Qi, et al. Edge Computing Paradigm: State of the Art and Research Challenges. *IEEE Communications Magazine*, 2017.
- [25]. Xu, Xiong, et al. Latency-Aware Edge Computing for Real-Time Financial Data Analytics. *IEEE Transactions on Services Computing*, 2021. Real-time Economic and Financial Decision Making (8)
- [26]. Xu, Zongpeng, et al. Real-time Big Data Analytics for Financial Applications. *IEEE Transactions on Big Data*, 2019.
- [27]. Lin, Yu-Chee, et al. Real-time Decision Making for Financial Markets Using IoT and Edge Computing. *IEEE Transactions on Industrial Informatics*, 2020.
- [28]. Li, Yong, et al. Real-time Analytics for Algorithmic Trading Using Edge Computing. *IEEE Transactions on Computational Social Systems*, 2021.

- [29]. Wang, Pei, et al. Edge-enabled Real-time Risk Management in Financial Services. ACM Transactions on Internet Technology, 2022.
- [30]. Zhao, Liang, et al. Real-time Economic Forecasting Using Edge Computing. IEEE Access, 2021.
- [31]. Chen, Liang, et al. Machine Learning at the Edge for Real-time Financial Decisions. IEEE Access, 2020.
- [32]. Chen, Wei, et al. Distributed Edge Analytics for Economic Data Streams. IEEE Internet of Things Journal, 2022.
- [33]. Chen, Xiang, et al. Low-latency Financial Data Processing in Edge Environments. IEEE Transactions on Emerging Topics in Computing, 2022.Low-latency Systems & Networking.
- [34]. Guo, Xiaohu, et al. Low-latency Communication for Edge Computing in 5G Networks. IEEE Wireless Communications, 2018.
- [35]. Wang, Jian, et al. Ultra-Reliable Low-Latency Edge Computing Networks for Industrial IoT. IEEE Transactions on Industrial Informatics, 2020.
- [36]. Cao, Zhitao, et al. Latency Minimization for Real-time Edge Computing. IEEE Transactions on Vehicular Technology, 2019.
- [37]. Zhang, Kai, et al. Network Slicing for Low Latency Edge Computing. IEEE Journal on Selected Areas in Communications, 2019.Taleb, Tarik, et al. Mobile Edge Computing: A Key Technology Towards 5G. IEEE Communications Magazine, 2017.
- [38]. Liu, Jie, et al. Low-Latency Data Processing at the Edge for Financial Applications. IEEE Transactions on Cloud Computing, 2021.Edge AI / Edge Analytics.
- [39]. Zhou, Ziyuan, et al. Edge Intelligence: Paving the Last Mile of AI with Edge Computing. Proceedings of the IEEE, 2020.
- [40]. Chen, Min, et al. AI at the Edge: Real-time Decision Making for Economic Systems. IEEE Network, 2021.
- [41]. Wang, Shuai, et al. Federated Learning and Edge Analytics for Financial Markets. IEEE Transactions on Neural Networks and Learning Systems, 2022.
- [42]. Ren, Jie, et al. Edge Computing-Enabled Real-Time Financial Forecasting. IEEE Internet of Things Journal, 2022.
- [43]. Li, Kezhi, et al. Real-time Edge AI for Stock Market Predictions. IEEE Transactions on Big Data, 2022.

- [44]. He, Zhengbing, et al. Online Machine Learning at the Edge for Economic Decision Support. IEEE Transactions on Industrial Informatics, 2023. Industry Reports & Whitepapers.
- [45]. Cisco. Cisco Edge Computing Solutions for Real-Time Analytics. 2022.
- [46]. IBM. Driving Real-Time Business Decisions Through Edge Analytics. 2021.
- [47]. Gartner. Market Guide for Edge Computing. 2023.
- [48]. McKinsey. Edge Computing in Financial Services: Unlocking Real-time Decisions. 2022.
- [49]. Accenture. Edge Intelligence for Real-time Economic Decisions. 2023. Books & Chapters.
- [50]. Yu, Weisong, et al. Edge Computing: Models, Technologies and Applications. Springer, 2019.
- [51]. Satyanarayanan, Mahadev. Edge Computing: A Primer. Cambridge University Press, 2020.
- [52]. Wang, Jian, et al. Real-time Systems and Edge Computing. Wiley, 2021.
- [53]. Chen, Min, et al. AI and Edge Computing for Finance. Springer, 2022.
- [54]. Li, X., et al. Financial Technology and Edge Computing. Routledge, 2023.
- [55]. Satyanarayanan, M., et al. Edge Analytics in the Internet of Things. IEEE Pervasive Computing, 2017.
- [56]. Bonomi, F., et al. Fog Computing and Its Role in the Internet of Things. MCC Workshop, 2012.
- [57]. Deng, Shancang, et al. Edge Computing Security: State of the Art and Challenges. IEEE Communications Surveys & Tutorials, 2019.
- [58]. Varghese, Blessen, et al. Edge Computing: Principles and Practice. IEEE Computer, 2020.
- [59]. Mach, P., Becvar, Z. Mobile Edge Computing: A Survey on Architecture and Computation Offloading. IEEE Communications Surveys & Tutorials, 2017.
- [60]. Munir, S., et al. Real-time Big Data Stream Processing in IoT: Edge Computing and Cloudlet Technologies. IEEE Access, 2020.
- [61]. Hassan, W., et al. Real-time Applications in Edge Computing: Architectures and Challenges. IEEE Communications Magazine, 2020.
- [62]. Wang, S., et al. Real-time Big Data Processing and Analytics on Edge Computing. Future Generation Computer Systems, 2021.
- [63]. Lin, K., et al. Latency-Aware Task Offloading and Resource Allocation for Edge Computing Systems. IEEE Transactions on Vehicular Technology, 2021.
- [64]. Hu, Y., et al. Mobile Edge Computing: Challenges and Future Directions. IEEE Wireless Communications, 2018.

- [65]. Yousefpour, A., et al. All One Needs to Know about Fog Computing and Related Edge Computing Paradigms. *Journal of Systems Architecture*, 2019.
- [66]. Shi, W., et al. Edge Computing: Vision and Challenges. *IEEE IoT Journal*, 2016.
- [67]. Mach, P., et al. Edge Computing for Real-time Data Analytics: A Survey. *ACM Computing Surveys*, 2020.
- [68]. Chen, M., et al. Distributed Real-Time Processing at the Edge: Emerging Trends and Open Issues. *IEEE Network*, 2020.
- [69]. Xie, S., et al. Latency Minimization in Real-time Edge Computing. *IEEE Transactions on Mobile Computing*, 2021 Financial and Economic Real-time Decision Making.
- [70]. Dong, Y., et al. Real-time Algorithmic Trading via Edge Computing Systems. *IEEE Transactions on Computational Social Systems*, 2021.
- [71]. Chen, X., et al. Edge-enabled Financial Fraud Detection in Real Time. *IEEE Transactions on Information Forensics and Security*, 2022.
- [72]. Lin, Y., et al. Real-time Financial Market Forecasting with Edge-based Deep Learning. *IEEE Transactions on Big Data*, 2022.
- [73]. Li, W., et al. Edge Computing for High-frequency Trading: Low Latency Strategies. *IEEE Access*, 2023.
- [74]. Wang, Z., et al. Real-time Risk Assessment for Financial Markets using Edge Analytics. *ACM Transactions on Internet Technology*, 2023.
- [75]. Zhang, Q., et al. Distributed Edge Analytics for Real-time Credit Scoring. *IEEE Internet of Things Journal*, 2022.
- [76]. He, J., et al. Real-time Economic Decision Support via Edge Computing and AI. *IEEE Transactions on Industrial Informatics*, 2023.
- [77]. Li, S., et al. Edge Computing for Real-time Market Microstructure Analysis. *Journal of Financial Data Science*, 2023.
- [78]. Zhao, J., et al. Real-time Portfolio Optimization using Edge Computing. *IEEE Transactions on Systems, Man, and Cybernetics*, 2022.
- [79]. Xu, H., et al. Edge-enabled Real-time Financial Data Stream Processing. *IEEE Transactions on Parallel and Distributed Systems*, 2023. Edge AI and Machine Learning.
- [80]. Liu, J., et al. Real-time Machine Learning on Edge Devices for Economic Forecasting. *IEEE Transactions on Neural Networks and Learning Systems*, 2022.